

A
MAJOR PROJECT REPORT ON
**FPGA-Based Automated Irrigation System Using LoRa
Communication**

*Project report submitted in partial fulfilment of the requirements for the award
of the degree*

BACHELOR TECHNOLOGY
in
ELECTRONICS & COMMUNICATION ENGINEERING

Submitted by

Ch. Jayanth	(221FA05179)
B. Revanth	(221FA05183)
M.V. Praveen Kumar	(221FA05191)
R. S. S. Prabhas	(221FA05210)

Under the guidance of

Dr. M.Sarada

Professor



Department of Electronics and Communication Engineering
School of Electrical, Electronics and Communication Engineering
Vignan's Foundation for Science, Technology and Research
(Deemed to be University)

Vadlamudi, Guntur, Andhra Pradesh-522213, India

MAY – 2026



CERTIFICATE

This is to certify that the major project entitled “**FPGA-Based Automated Irrigation System Using LoRa Communication**” being submitted by Ch. Jayanth (221FA05179), B. Revanth (221FA05183), M.V. Praveen Kumar (221FA05191), and R.S.S. Prabhas (221FA05210) in partial fulfilment of Bachelor of Technology in the Department of Electronics and Communication Engineering, Vignan’s Foundation For Science Technology & Research (Deemed to be University), Vadlamudi, Guntur District, Andhra Pradesh, India, is a bonafide work carried out by them under my guidance and supervision.

Dr. M. Sarada

Professor

Department of ECE

Date:

VFSTR, Vadlamudi

Andhra Pradesh-522213

India

Dr. M. Sarada

Professor

Department of ECE

Date:

VFSTR, Vadlamudi

Andhra Pradesh-522213

India

External Examiner

DECLARATION

We hereby declare that our project work described in the major project titled “**FPGA-Based Automated Irrigation System Using LoRa Communication**” which is being submitted by us for the partial fulfilment in the department of Electronics and Communication Engineering, Vignan’s Foundation for Science, Technology and Research (Deemed to be University), Vadlamudi, Guntur, Andhra Pradesh, and the result of investigations are carried out by us under the guidance of **Dr. M. Sarada**

Ch. Jayanth	(221FA05179)
B. Revanth	(221FA05183)
M.V. Praveen Kumar	(221FA05191)
R. S. S. Prabhas	(221FA05210)

ACKNOWLEDGEMENT

It is our immense pleasure to say thanks to all the people who are involved directly or indirectly in accomplishing this thesis work. First and foremost, we would like to thank to the *Almighty God* who gave us strength, encouragement and wisdom to carry out our major project work. He always showered blessings which enable us to reach this stage in my life

We are honored to express our deepest gratitude to my supervisor **Dr. M. Sarada**, Department of Electronics Communication Engineering, VFSTR, Guntur for their discussions, encouragement and valuable advice and guidance throughout our major project work. With their kindness, dedication, hard-work and habits to do something new have been always source of inspiration to me. They have always given me the freedom to explore new things and ideas which has proven a great milestone throughout the work in association with them. Their constant motivation, suggestions and encouragement brought me to the right track during the difficult time of the thesis work. Regular talks, discussions with them made research environment more fruitful and comfortable during research time. They gave invaluable suggestions, corrections during writing or communicating research articles keep me one step ahead in this area..

We are obliged to **Dr. M.Sarada** (HoD, Department of Electronics and Communication Engineering), **Dr. Y.Ravisekar** (Dean, Department of Electrical, Electronics and Communication Engineering) and the major Project Review Committee members, **Dr. P.J. Reginald**, **Dr. G.S.R.Satya Narayana** for their periodic suggestions during reviews.

We would also like to thank all faculty members and staff of the Department of Electronics and Communication Engineering, School of Electrical, Electronics and Communication Engineering VFSTR, Guntur, for their help in various ways for the completion of this major project report. The financial support throughout my **B.Tech.**, studies provided by our institute is gratefully acknowledged.

Name of the Student(s)

Ch. Jayanth

B. Revanth

M.V. Praveen Kumar

R.S.S. Prabhas

Major Project Experience Information

Student Group	Ch. Jayanth (221FA05179)	B. Revanth (221FA05183)	M. V. Praveen Kumar (221FA05191)	R. S. S. Prabhas (221FA05210)
---------------	-----------------------------	----------------------------	--	-------------------------------------

Project Title	FPGA-Based Automated Irrigation System Using LoRa Communication
Programme Concentration Area	FPGA Design, Embedded Systems, Smart Agriculture, IoT

Constraints: Examples	
Economic	Reduces water wastage and labour costs in irrigation systems
Environmental	Promotes efficient utilisation of water resources and sustainable farming
Sustainability	Supports long-term agricultural sustainability through automated irrigation
Manufacturability	The system can be implemented using commercially available sensors and FPGA boards
Ethical	Ensures responsible usage of natural resources
Health and Safety	Prevents electrical and motor damage through safe water-level monitoring
Social	Supports farmers by reducing manual effort and improving irrigation efficiency
Political	NA
Others	Supports precision agriculture applications

Standards/Tools	
IEEE 1364-2005	Verilog HDL standard used for RTL coding and hardware modeling
Xilinx Vivado 2018.1	Used for synthesis, implementation, timing analysis, and bitstream generation for FPGA
IEEE 1149.1-1990	JTAG boundary scan standard used for testing and debugging FPGA hardware
ModelSim	Used for functional simulation and waveform verification of Verilog designs
FPGA Development Board	Used for real-time hardware implementation and testing of the design

Previous Course Required for Major Project	Digital Electronics, Verilog HDL, Embedded Systems, Microprocessors and Microcontrollers, FPGA Design
--	---

Dr. M. SARADA
Professor
Department of ECE

Dr. M SARADA
Professor
Department of ECE

Major Project SDG Mapping

SDG Number	SDG Title	Project Relevance
1	No Poverty	NA
2	Zero Hunger	NA
3	Good Health and Well-being	NA
4	Quality Education	Promotes learning and application of smart agricultural technologies
5	Gender Equality	NA
6	Clean Water and Sanitation	NA
7	Affordable and Clean Energy	Can be integrated with renewable energy sources such as solar power
8	Decent Work and Economic Growth	Reduces labor dependency and improves agricultural productivity
9	Industry, Innovation and Infrastructure	Encourages innovation using FPGA and automation technologies
10	Reduced Inequality	Supports small-scale farmers through affordable automation solutions
11	Sustainable Cities and Communities	NA
12	Responsible Consumption and Production	Ensures efficient utilization of water resources
13	Climate Action	Supports sustainable farming and water conservation practices
14	Life Below Water	NA
15	Life on Land	NA
16	Peace, Justice and Strong Institutions	NA
17	Partnerships for the Goals	Encourages integration of engineering and agriculture for sustainable development

Dr. M. SARADA
Professor
Department of ECE

Dr. M SARADA
Professor
Department of ECE

CONTENTS

Certification.....	i
Declaration.....	ii
Acknowledgements	iii
Major Project Experience Information.....	v
Major Project Sdg Mapping.....	vi
List Of Tables.....	x
List Of Figures.....	xi
Abstract.....	xvii
Chapter 1: Introduction	
1.1 Background.....	1
1.2 Smart Irrigation Systems.....	3
1.3 Role of FPGA in Automation Systems.....	6
1.4 System Architecture Overview.....	10
1.5 Problem Statement.....	10
1.6 Proposed System.....	11
1.7 Motivation.....	13
1.8 Objectives.....	15
1.9 Scope of the Work.....	15
1.10 Organization of the Report.....	15
Chapter 2: Literature Review	
2.1 Introduction.....	17
2.2 Conventional Irrigation Systems.....	19
2.3 Microcontroller-Based Irrigation Systems.....	22
2.4 Iot-Based Smart Irrigation Systems	25
2.5 FPGA-Based Control Systems	28
2.6 Comparative Analysis of Existing Systems.....	30
2.7 Research Gap.....	31
Chapter 3: System Design	
3.1 Introduction.....	33
3.2 Overall System Architecture	34
3.3 Block Diagram of the System	35
3.4 Description of System Components	36
3.5 Working Principle of the System.....	37
3.6 Design Considerations.....	38
Chapter 4: Hardware Design	
4.1 Introduction.....	39
4.2 Overview of Hardware Components.....	39
4.3 Soil Moisture Sensor	40
4.4 Arduino-Based ADC Interface	41

	4.5	FPGA Development Board.....	42
	4.6	Motor and Relay Driver Circuit.....	43
	4.7	Power Supply Unit.....	44
	4.8	Hardware Integration.....	44
Chapter 5:Software Design			
	5.1	Introduction.....	46
	5.2	Design Methodology	46
	5.3	Verilog Implementation	46
	5.4	Simulation and Verification	62
Chapter 6: Results & Analysis			
	6.1	Introduction.....	63
	6.2	Simulation Setup	65
	6.3	Simulation Waveforms.....	66
	6.4	FPGA Implementation.....	68
	6.5	Performance Analysis.....	69
	6.6	Comparison with Existing Systems.....	69
	6.7	Limitations Observed.....	70
Chapter 7: Limitations			
	7.1	Hardware and Sensor Limitations	71
	7.2	System-Level and Functional Limitations	72
Chapter 8: Summary, Conclusion & Future Scope			
	8.1	Summary of the Work.....	74
	8.2	Conclusions.....	74
	8.3	Future Scope.....	75
References.....			76

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Conventional Irrigation Methods Based on Advantages and Disadvantages	21
2.2	Comparative Analysis of Irrigation Systems Based on Automation, Speed, Real-Time Capability, Cost, and Reliability.....	31
3.1	List of System Components and Their Functions in the Proposed Irrigation System.....	35
4.1	Hardware Components Specification and Their Functional Description.....	40
6.1	Test Cases for System Simulation with Expected Motor Control Output.....	66
6.2	Analysis of Input Conditions, Waveform Behavior, and Motor Output Response.....	67
6.3	Performance Comparison of Microcontroller, IoT, and FPGA-Based Systems...	70
8.1	Hardware Limitations and Their Impact on System Performance.....	72

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Circuit Diagram of Soil Moisture Sensor with Signal Conditioning and Alert System	3
1.2	Internal Architecture of FPGA Showing Port Logic, Timing Control, and Management Modules.....	4
1.3	Block Diagram of FPGA-Based Automated Irrigation System Using LoRa Communication.....	4
1.4	Traditional Flood Irrigation Method in Agricultural Field.....	7
2.1	Circuit Diagram of Arduino-Based Irrigation System with Sensors, Relay Control, and Display Interface.....	23
2.2	IoT-Based Smart Irrigation System Architecture with Cloud and Remote Monitoring.....	26
2.3	Block Diagram of Multi-Channel Data Acquisition and Processing Architecture.....	28
4.1	Soil Moisture Sensor Module Used for Measuring Soil Water Content.....	41
4.2	Arduino Uno with ADC input channels.....	42
4.3	ZedBoard (Zynq-7000 SoC) FPGA Development Board with Peripheral Interfaces.....	43
6.1	Simulation Waveform Showing Motor Control Output Based on Input Signal Conditions.....	66
6.2	Experimental Setup of FPGA-Based Irrigation System with ZedBoard, Arduino, and Sensor Interface	68
6.3	Experimental Setup Showing Motor ON Condition Based on Sensor Input.....	68

ABSTRACT

Efficient water management has become a critical challenge in modern agriculture, particularly in developing countries like India, where irrigation accounts for a significant portion of water consumption. Traditional irrigation practices such as manual watering and time-based irrigation often lead to inefficient utilization of water resources. These methods depend heavily on human intervention and lack real-time monitoring, resulting in issues such as over-irrigation, under-irrigation, uneven water distribution, and increased labor requirements. Such limitations not only waste valuable water resources but also negatively impact crop productivity and soil health. Therefore, there is a growing need for an intelligent, automated, and reliable irrigation system that can optimize water usage while reducing human effort.

This project presents the design and implementation of an FPGA-Based Automated Water Management System for Smart Agriculture. The proposed system integrates sensing, communication, and processing units to achieve real-time monitoring and automatic control of irrigation. Soil moisture and water level sensors are used to continuously monitor environmental conditions in the agricultural field. Since these sensors generate analog signals, an Arduino Uno is employed to perform Analog-to-Digital Conversion (ADC), converting the sensor outputs into digital data suitable for further processing.

To enable wireless communication over long distances, LoRa (Long Range) technology is used. A LoRa transmitter module connected to the Arduino sends the processed sensor data to a LoRa receiver interfaced with the Zynq-7000 System on Chip (SoC), implemented on a ZedBoard FPGA platform. The FPGA acts as the central processing unit of the system and executes control logic designed using Verilog Hardware Description Language (HDL). The use of FPGA technology provides high-speed parallel processing, low latency, and reliable real-time performance, making it suitable for critical control applications like irrigation.

The system operates based on predefined threshold values for soil moisture and water level. When the soil moisture falls below the required threshold and sufficient water is available in the reservoir, the FPGA generates a control signal to activate the irrigation motor. Conversely, when the desired moisture level is achieved or when the water level is insufficient, the motor is

automatically turned off. This closed-loop control mechanism ensures efficient water usage and prevents wastage.

The proposed system was designed, simulated, and implemented successfully. Simulation results were obtained using FPGA design tools, and experimental validation was carried out using the ZedBoard hardware platform. The results demonstrate that the system responds quickly to changes in environmental conditions and provides accurate control over irrigation. The integration of LoRa communication ensures reliable data transmission even in remote agricultural areas with limited connectivity.

Overall, the proposed FPGA-based irrigation system offers significant advantages, including reduced water wastage, improved crop yield, minimal human intervention, and enhanced system reliability. The modular design also allows for future expansion, such as integration with additional sensors or intelligent decision-making algorithms. This work contributes toward the development of efficient and sustainable smart agriculture solutions.

Chapter-1: INTRODUCTION

1.1 Background

Agriculture plays a vital role in sustaining human life and forms the backbone of many developing economies, especially in countries like India where a significant portion of the population depends on farming for livelihood. Efficient water management is one of the most critical aspects of agriculture, as crop productivity is directly influenced by irrigation practices. Traditional irrigation methods such as flood irrigation, manual watering, and fixed-schedule irrigation have been widely used for decades; however, these conventional systems suffer from several limitations that reduce efficiency and lead to resource wastage.

One of the primary challenges in conventional irrigation systems is the lack of real-time monitoring and control. Farmers typically rely on manual observation or fixed time intervals to irrigate fields, which often results in either over-irrigation or under-irrigation. Over-irrigation leads to water wastage, soil nutrient leaching, and increased salinity, while under-irrigation can stress crops and reduce yield. Studies such as those by Miranda et al. highlight that traditional irrigation lacks site-specific adaptability, making it inefficient for varying soil and crop conditions [5]. Similarly, early microcomputer-based irrigation systems attempted to address automation but were limited by technology constraints and lack of sensor integration [6].

Another major limitation is the inefficient utilization of water resources. In conventional methods like flood irrigation, a large portion of water is lost due to evaporation, runoff, and deep percolation. This inefficiency is particularly problematic in regions facing water scarcity. Gutiérrez et al. demonstrated that automated irrigation systems using wireless sensor networks significantly improve water usage efficiency compared to traditional methods [1][3]. The absence of such intelligent systems in conventional irrigation contributes to unsustainable agricultural practices.

Labor dependency is also a critical issue. Traditional irrigation systems require continuous human involvement for monitoring soil conditions, operating pumps, and managing water distribution. This not only increases labor costs but also introduces human error into the system. Manual decision-making often lacks precision, leading to inconsistent irrigation patterns. Zazueta and Smajstrla emphasized that earlier irrigation

control methods required significant human supervision, reducing overall system efficiency [6].

Conventional irrigation systems also fail to incorporate environmental and climatic variations. Factors such as soil moisture, temperature, humidity, and rainfall are rarely considered in manual irrigation practices. As a result, irrigation schedules remain static and do not respond dynamically to changing environmental conditions. Modern research shows that integrating sensors and automated controllers can optimize irrigation by adapting to real-time field data, as seen in wireless and solar-powered irrigation systems [2]. Without such adaptability, conventional systems cannot achieve optimal crop growth.

Energy inefficiency is another drawback. Traditional irrigation systems often rely on continuous operation of pumps, which may not be energy optimized. In many rural areas, this leads to excessive electricity consumption or dependence on unreliable power sources. The introduction of solar-powered irrigation systems has been shown to reduce energy costs and improve sustainability, highlighting a major gap in conventional approaches [2].

Additionally, conventional systems lack scalability and precision. Techniques such as drip irrigation, when not automated, still depend on manual control and cannot adjust water flow precisely based on crop requirements. Intelligent approaches like linear programming-based irrigation and automated drip systems demonstrate improved precision and resource management, which conventional systems fail to achieve [4].

The absence of integration with modern technologies such as wireless communication, IoT, and embedded systems further limits conventional irrigation. Advanced systems using GPRS modules and wireless sensor networks enable remote monitoring and control, allowing farmers to manage irrigation efficiently even from distant locations [1]. Similarly, platforms like Arduino have enabled low-cost automation and real-time data processing in irrigation systems, making smart agriculture more accessible [8][9][10][11]. Conventional systems, however, remain isolated and lack such connectivity.

In conclusion, while traditional irrigation methods have served agriculture for many

years, they are increasingly inadequate in meeting modern agricultural demands. The major limitations include lack of real-time monitoring, inefficient water usage, high labor dependency, inability to adapt to environmental changes, energy inefficiency, and absence of technological integration. These challenges highlight the need for transitioning toward automated and intelligent irrigation systems that utilize sensors, wireless communication, and embedded platforms. Such advancements not only improve water efficiency and crop yield but also contribute to sustainable agricultural practices, addressing the growing global demand for food and resource conservation.

1.2 Smart Irrigation Systems

Smart irrigation systems represent a significant advancement in modern agriculture by integrating sensing technologies, automation, and communication networks to optimize water usage and improve crop productivity. Unlike conventional irrigation methods, which rely on manual control or fixed schedules, smart irrigation systems dynamically adjust water delivery based on real-time environmental and soil conditions. These systems are designed to enhance efficiency, reduce resource wastage, and support sustainable agricultural practices.

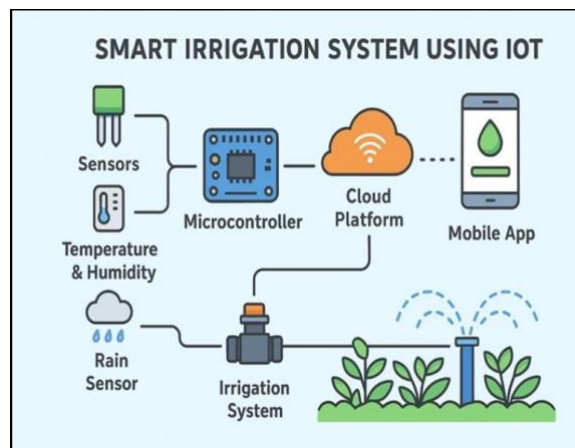


Fig 1.1: IoT-Based Smart Irrigation System Architecture with Cloud and Mobile Application Integration

One of the core components of smart irrigation systems is the use of sensors to monitor key parameters such as soil moisture, temperature, humidity, and water levels. These sensors provide continuous feedback, enabling precise irrigation decisions. For instance, the work by Gutiérrez et al. demonstrates the use of wireless sensor networks (WSNs) combined with GPRS modules to collect field data and transmit it to a central system for intelligent decision-making [1][3]. This real-time monitoring ensures that

water is supplied only when required, preventing both over-irrigation and under-irrigation.



Fig 1.2: Soil Moisture Measurement Using Portable Sensor in Agricultural Field

Automation is another critical feature of smart irrigation systems. Controllers process sensor data and automatically operate irrigation components such as pumps and valves. Early developments in this area include microcomputer-based irrigation control systems, which laid the foundation for modern automated solutions [6]. More advanced systems, such as those proposed by Miranda et al., incorporate autonomous controllers capable of site-specific irrigation management, thereby improving water distribution efficiency across different zones of a field [5].

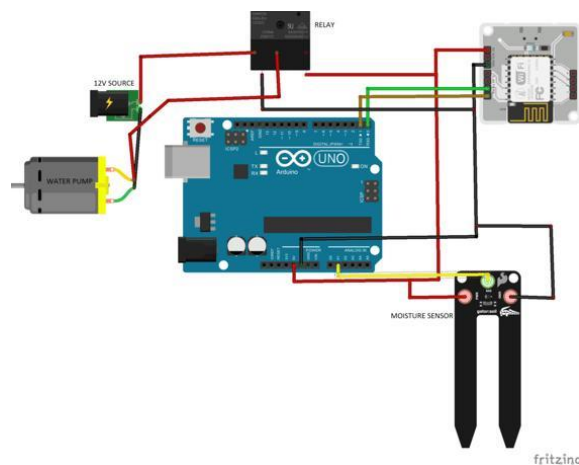


Fig 1.3: Circuit Diagram of Arduino-Based Irrigation System with Soil Moisture Sensor, Relay, and Water Pump

Energy efficiency is also addressed in smart irrigation through the integration of renewable energy sources. Solar-powered irrigation systems have gained popularity, particularly in remote and rural areas where conventional power supply is unreliable.

The study by JiaUddin et al. highlights the effectiveness of solar energy in powering automated irrigation systems, reducing operational costs and promoting eco-friendly practices [2]. Such systems ensure uninterrupted irrigation while minimizing dependence on grid electricity.

Communication technologies play a vital role in enhancing the functionality of smart irrigation systems. Wireless communication methods, including GPRS and IoT-based platforms, enable remote monitoring and control. Farmers can access system data and manage irrigation operations from distant locations, improving convenience and decision-making. The integration of wireless networks in irrigation systems, as discussed in [1], allows seamless data transfer and real-time alerts, making the system more responsive and reliable.

Another important aspect of smart irrigation is precision agriculture. Techniques such as automated drip irrigation and linear programming-based water allocation help deliver the exact amount of water required by crops. Giri and Wavhal proposed an intelligent drip irrigation system that uses optimization techniques to enhance water use efficiency and crop yield [4]. This level of precision is not achievable with traditional irrigation methods and significantly contributes to resource conservation.

Embedded systems, particularly platforms like Arduino, have made smart irrigation systems more accessible and cost-effective. Arduino-based systems are widely used for prototyping and implementing irrigation automation due to their simplicity, flexibility, and low cost. According to Leo Louis, Arduino serves as an effective tool for research and development in automation applications [8]. The official Arduino resources also emphasize its ease of integration with sensors and actuators, making it ideal for smart irrigation projects [9][10][11].

In addition to water management, smart irrigation systems can be integrated with other advanced agricultural technologies. For example, machine vision systems for row-following agricultural machinery, as discussed by Astrand and Baerveldt, can complement irrigation systems by enabling precise field navigation and crop monitoring [7]. Such integrations contribute to the development of fully automated and intelligent farming systems.

Despite their advantages, smart irrigation systems also face certain challenges, including initial installation cost, technical complexity, and the need for proper maintenance. However, the long-term benefits in terms of water savings, increased crop yield, and reduced labor requirements outweigh these limitations. As technology continues to evolve, the cost of components is decreasing, making these systems more affordable for small and medium-scale farmers.

In conclusion, smart irrigation systems are transforming agriculture by introducing automation, real-time monitoring, and intelligent decision-making. By leveraging technologies such as wireless sensor networks, embedded systems, renewable energy, and communication platforms, these systems address the limitations of conventional irrigation and promote sustainable farming practices. The research and developments presented in the referenced works clearly demonstrate the potential of smart irrigation to improve efficiency, conserve resources, and support the growing global demand for food production.

1.3 Role of FPGA in Automation Systems

Field Programmable Gate Arrays (FPGAs) have emerged as a powerful technology in the design and implementation of modern automation systems, offering high speed, parallel processing capability, reconfigurability, and real-time performance. In the context of agricultural automation—particularly smart irrigation systems—FPGAs play a crucial role in enabling efficient data processing, control logic implementation, and reliable system operation. While earlier irrigation systems relied on microcontrollers and basic automation techniques, the increasing complexity of modern systems has driven the adoption of FPGA-based solutions for enhanced performance and flexibility.

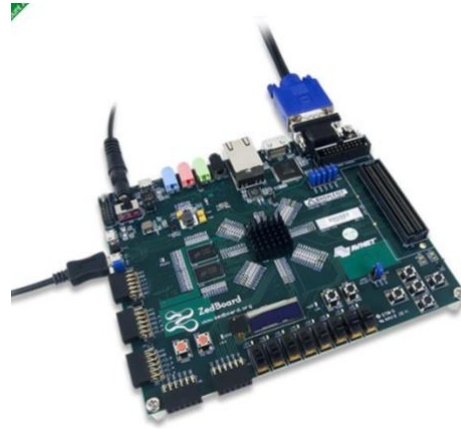


Fig 1.4: ZedBoard (Zynq-7000 FPGA) Used for Real-Time Irrigation Control

One of the key advantages of FPGAs in automation systems is their ability to execute multiple operations in parallel. Unlike traditional microcontroller-based systems, which process instructions sequentially, FPGAs can handle multiple sensor inputs and control outputs simultaneously. This is particularly beneficial in irrigation systems where parameters such as soil moisture, temperature, humidity, and water level must be continuously monitored and processed in real time. Studies on wireless sensor-based irrigation systems highlight the importance of real-time data acquisition and processing for efficient water management [1][3]. FPGAs enable this by implementing parallel data paths, reducing latency and improving system responsiveness.

Another significant role of FPGAs is in real-time control and decision-making. Automation systems require immediate responses to changing environmental conditions, such as turning on a water pump when soil moisture falls below a threshold. Early systems using microcomputer-based control demonstrated basic automation capabilities but were limited in speed and scalability [6]. FPGAs overcome these limitations by implementing hardware-level control logic, allowing faster and deterministic responses. This makes them highly suitable for critical applications where timing accuracy is essential.

FPGAs also provide high flexibility and reconfigurability, which is a major advantage in research and development of automation systems. Designers can modify the functionality of an FPGA by updating the hardware description language (HDL) code without changing the physical hardware. This is particularly useful in smart irrigation systems, where control algorithms may need to be updated based on crop type, soil

Another emerging role of FPGAs is in supporting advanced technologies such as machine vision and intelligent automation. For example, vision-based agricultural systems for tasks like row following and crop monitoring require high-speed image processing capabilities [7]. FPGAs are well-suited for such applications due to their ability to handle large data streams and perform parallel computations efficiently. This opens the door for integrating irrigation systems with advanced monitoring and control mechanisms, leading to fully automated smart farming solutions.

While microcontrollers such as Arduino are widely used for low-cost automation and prototyping, they have limitations in terms of processing power and scalability. Arduino-based systems are effective for basic irrigation control and educational purposes [8][9][10][11], but as system complexity increases, FPGAs provide a more powerful alternative. They enable implementation of complex algorithms, multi-sensor data fusion, and high-speed control operations that are beyond the capabilities of simple microcontroller platforms.

Despite their advantages, FPGAs also present certain challenges, including higher initial cost, design complexity, and the need for expertise in hardware description languages such as Verilog or VHDL. However, with the increasing availability of development boards like the Zynq-7000 SoC, which combines FPGA fabric with embedded processors, these challenges are gradually being reduced. Such platforms provide a balance between hardware acceleration and software flexibility, making FPGA-based automation systems more accessible to researchers and developers.

In conclusion, FPGAs play a vital role in modern automation systems by providing high-speed, parallel processing, real-time control, and system flexibility. In agricultural applications such as smart irrigation, they enable efficient sensor data processing, precise control of actuators, and integration with advanced technologies like wireless communication and machine vision. Compared to traditional microcontroller-based systems, FPGA-based solutions offer superior performance, scalability, and reliability. As automation systems continue to evolve, the role of FPGAs will become increasingly significant in achieving intelligent, efficient, and sustainable solutions for agriculture and beyond.

1.4 Problem Statement

Despite the availability of modern irrigation techniques, many agricultural systems still rely on manual operation. This leads to inefficient water usage and increased labor requirements.

Farmers often irrigate fields based on assumptions rather than actual soil conditions. This results in excessive water consumption and reduced efficiency. Additionally, manual systems are not capable of responding quickly to changing environmental conditions.



Fig 1.4: Traditional Flood Irrigation Method in Agricultural Field

Another major issue is the lack of remote monitoring. Farmers need to be physically present in the field to operate irrigation systems, which is not always practical, especially for large farms.

These challenges highlight the need for an automated system that can monitor soil conditions in real-time and control irrigation accordingly. The system should also be capable of remote operation and should minimize human intervention.

1.5 Proposed System

A new automated irrigation system has been developed based on FPGAs (Field Programmable Gate Arrays) to resolve some of the restrictions of current systems.

Sensors are used to calculate both moisture levels in the soil and how much water is in the tank. This data is processed by an FPGA, which indicates whether or not to irrigate. When the soil moisture level goes below a predetermined threshold value, it activates an electric motor; if the soil moisture level does not fall below the threshold, the motor will remain inactive.

LoRa (Long Range) technology is incorporated into the automated irrigation system, allowing users to monitor their crops remotely over long distances by transmitting data from one location to another.

The new automated irrigation system is:

- capable of controlling irrigation in "real-time"
- an efficient way to manage water
- minimizes the amount of human labor required for irrigation
- a design that can be made larger for large fields

1.6 Advantages & Applications

Advantages

The proposed FPGA-based automated irrigation system offers several significant advantages over conventional irrigation methods:

1. Efficient Water Utilization

The system continuously monitors soil moisture levels and supplies water only when required. This minimizes water wastage and ensures optimal usage, which is especially important in regions facing water scarcity.

2. Reduction in Manual Labor

Automation of irrigation eliminates the need for constant human supervision. Farmers are not required to manually operate pumps, thereby reducing labor effort and operational costs.

3. Real-Time Processing Using FPGA

The use of the ZedBoard enables high-speed and parallel data processing. This ensures immediate response to sensor inputs, resulting in efficient and reliable irrigation control.

4. Improved Crop Productivity

By maintaining optimal soil moisture conditions, the system helps in enhancing crop growth and yield. Over-irrigation and under-irrigation are effectively avoided.

5. Low Power and Cost-Effective Operation

The system is designed to operate with minimal power consumption. Once deployed, it reduces recurring costs associated with manual irrigation practices.

6. Scalability and Flexibility

The design can be easily extended to support multiple sensors and larger agricultural fields. Additional features such as weather monitoring and automation logic can also be integrated.

7. Remote Monitoring Capability

With the integration of LoRa communication, the system enables long-range data transmission. This allows users to monitor irrigation status remotely without the need for physical presence in the field.

Applications

The developed system can be applied in various domains related to agriculture and water management:

1. Smart Agriculture Systems

The system is highly suitable for modern smart farming techniques where automation and data-driven decision-making are essential.

2. Irrigation in Water-Scarce Regions

In areas facing water shortages, the system ensures efficient water distribution, making it ideal for sustainable agriculture practices.

3. Greenhouses and Controlled Environments

The system can be used in greenhouse farming where precise control of soil moisture is required for optimal plant growth.

4. Large-Scale Agricultural Fields

Due to its scalability, the system can be implemented in large farms to automate irrigation over wide areas.

5. Remote Farming Locations

With LoRa-based communication, the system is effective in remote or rural areas where internet connectivity is limited.

6. Research and Development in Precision Agriculture

The system can be used as a prototype or base model for further research in precision agriculture and IoT-based farming solutions.

1.7 Motivation

The motivation behind this project arises from the urgent need to improve water efficiency and reduce human effort in modern agriculture. Agriculture remains highly dependent on natural resources, particularly water, and its effective management plays a crucial role in ensuring crop productivity and sustainability. In many regions, especially in developing countries, farmers continue to rely on traditional irrigation practices that are often inefficient, labor-intensive, and incapable of adapting to changing environmental conditions. As a result, there is a growing demand for intelligent systems that can optimize irrigation processes and support farmers in making better decisions.

Farmers today face several critical challenges that directly impact agricultural output. One of the major issues is water scarcity. With increasing population and climate change, the availability of freshwater resources is becoming limited. Conventional irrigation methods, such as flood irrigation, lead to excessive water wastage due to evaporation, runoff, and improper distribution. This not only reduces water availability for future use but also affects soil quality. In addition to water shortages, unpredictable rainfall patterns further complicate irrigation planning. Climate variability has made it difficult for farmers to rely on seasonal rainfall, leading to either drought conditions or excessive water supply, both of which can harm crops.

Another significant challenge is the shortage of labor in agricultural fields. Manual irrigation requires constant monitoring of soil conditions, water levels, and crop requirements. Farmers or laborers must physically operate pumps, open and close valves, and ensure that water is distributed evenly across the field. This process is time-consuming, inefficient, and prone to human error. In many cases, farmers are unable to provide timely irrigation due to labor unavailability, which directly affects crop health and yield. These limitations clearly indicate that manually operated irrigation systems are no longer sufficient to meet the demands of modern agriculture.

To address these issues, there is a need for an advanced irrigation system capable of making intelligent decisions based on real-time conditions. Such a system should be able to monitor parameters like soil moisture, temperature, humidity, and water levels,

and automatically control irrigation without human intervention. By adapting to changing environmental conditions, the system can ensure that crops receive the right amount of water at the right time, thereby improving efficiency and productivity. Automation not only reduces human effort but also minimizes resource wastage and enhances overall system reliability.

In this context, the use of Field Programmable Gate Arrays (FPGAs) offers a powerful solution for developing advanced automation systems. FPGAs provide high-speed processing and parallel execution capabilities, enabling the system to handle multiple sensor inputs simultaneously and make quick decisions. Unlike conventional microcontroller-based systems, which operate sequentially, FPGA-based systems can process data in real time with minimal delay. This is particularly important in irrigation systems, where immediate response to changing conditions is essential for optimal performance.

Furthermore, FPGAs offer high reliability and deterministic operation, making them suitable for critical agricultural applications. Since the control logic is implemented at the hardware level, FPGA-based systems are less prone to software-related issues such as delays or crashes. They also provide flexibility, allowing designers to modify system functionality by updating the hardware description code without changing the physical components. This adaptability is beneficial for customizing irrigation strategies based on different crops, soil types, and environmental conditions.

In addition to performance and reliability, FPGA-based systems support integration with modern technologies such as sensors, wireless communication modules, and energy-efficient power sources. This enables the development of a fully automated and intelligent irrigation system that can be monitored and controlled remotely. By combining real-time data processing with automated control, the proposed system can significantly reduce water wastage, lower labor dependency, and improve agricultural productivity.

In conclusion, the motivation for this project is driven by the need to overcome the limitations of traditional irrigation systems and address the challenges faced by modern farmers. By leveraging FPGA technology, the proposed system aims to provide a fast, reliable, and efficient solution for smart irrigation. This approach not only enhances

water management but also contributes to sustainable agriculture, ensuring better resource utilization and improved crop yield in the long run.

1.8 Objectives

The primary objective of this project is to design and implement an FPGA-Based Automated Water Management System for Smart Agriculture that can efficiently control irrigation using real-time environmental data.

The specific objectives include:

- To monitor soil moisture and water level conditions using appropriate sensors
- To convert analog sensor outputs into digital signals using Arduino Uno
- To design and implement control logic using Verilog HDL on Zynq-7000 SoC
- To automate the operation of a water motor based on predefined threshold conditions
- To minimize water wastage and reduce human intervention

1.9 Scope of the Work

The scope of this project includes the design and implementation of a sensor-based automated irrigation system using FPGA.

The system focuses on:

- Real-time soil monitoring
- Automatic motor control
- Efficient water usage
- The system can be extended in the future by integrating:
- IoT-based monitoring
- Mobile application control
- AI-based irrigation prediction

1.10 Organization of the Report

The remainder of this report is organized as follows:

- Chapter 2 presents a comprehensive review of existing literature related to automated irrigation systems.
- Chapter 3 describes the overall system design and methodology.
- Chapter 4 explains the hardware components and implementation details.

- Chapter 5 discusses the software design and Verilog implementation.
- Chapter 6 presents the results and analysis of the system.
- Chapter 7 concludes the report and outlines future scope for further development.

Chapter-2: LITERATURE SURVEY

2.1 Introduction

The rapid advancement of technology in agriculture has brought about significant improvements in productivity, efficiency, and sustainability. Over the past few decades, the integration of automation, embedded systems, wireless communication, and renewable energy into agricultural practices has transformed traditional farming into a more intelligent and data-driven process. These technological developments have enabled farmers to optimize resource utilization, reduce operational costs, and improve crop yield, thereby addressing many of the challenges faced in conventional agriculture.

One of the major results of technological advancement in agriculture is the development of smart irrigation systems that utilize real-time data for decision-making. Systems based on wireless sensor networks (WSNs) and communication technologies such as GPRS allow continuous monitoring of environmental parameters like soil moisture, temperature, and humidity. For example, the work by Gutiérrez et al. demonstrates how integrating WSNs with irrigation systems enables precise water management and remote monitoring, significantly improving efficiency compared to traditional approaches [1][3]. These advancements reduce water wastage and ensure that crops receive the required amount of water at the right time.

Another significant transformation is the shift from manual and labor-intensive farming practices to automated and intelligent systems. Earlier irrigation systems relied heavily on human intervention, requiring farmers to manually operate pumps and monitor field conditions. However, modern systems incorporate automation through microcontrollers, FPGAs, and embedded platforms, reducing labor dependency and human error. Zazueta and Smajstrla highlighted the early use of microcomputer-based control systems, which marked the beginning of automation in irrigation, although these systems were limited in capability compared to today's technologies [6]. The evolution from such basic systems to advanced automated solutions reflects the profound impact of technological progress.

Over the past few decades, various irrigation techniques have been developed to improve water efficiency and crop productivity. Traditional methods such as flood

irrigation have gradually been replaced or supplemented by advanced techniques like drip irrigation, sprinkler systems, and intelligent irrigation systems. Drip irrigation, in particular, delivers water directly to the root zone of plants, minimizing losses due to evaporation and runoff. Further advancements include automated drip irrigation systems that use optimization techniques, such as linear programming, to allocate water efficiently based on crop requirements, as discussed by Giri and Wavhal [4]. These modern techniques represent a significant improvement over older methods, enabling precise and efficient water usage.

In addition to water management, technological advancements have also introduced energy-efficient solutions in agriculture. Solar-powered irrigation systems have emerged as a sustainable alternative to conventional electricity-based systems, especially in rural areas with limited power supply. The study by JiaUddin et al. demonstrates how solar energy can be effectively used to power automated irrigation systems, reducing operational costs and promoting environmentally friendly practices [2]. This integration of renewable energy with automation further enhances the sustainability of modern agriculture.

Another important transformation is the incorporation of advanced technologies such as machine vision and intelligent control systems. Vision-based systems for agricultural machinery, such as row-following systems, improve precision in farming operations and reduce manual effort [7]. These technologies, when combined with automated irrigation systems, contribute to the development of fully integrated smart farming solutions.

Despite these advancements, traditional irrigation methods still persist in many regions and continue to exhibit several limitations. One of the major drawbacks of traditional systems is the lack of real-time monitoring and adaptability. Farmers typically rely on fixed schedules or manual observation, which does not account for variations in soil moisture or weather conditions. This often results in inefficient water usage, either through over-irrigation or under-irrigation. Miranda et al. emphasized the importance of site-specific irrigation management, which is not achievable with conventional methods [5].

Traditional irrigation methods also lack precision and control. Water distribution in methods like flood irrigation is uneven, leading to wastage and reduced effectiveness.

Furthermore, these systems do not incorporate modern communication technologies, making remote monitoring and control impossible. As a result, farmers must be physically present to manage irrigation, increasing labor requirements and operational challenges.

Additionally, traditional systems are not energy-efficient and often rely on continuous operation of pumps, leading to higher energy consumption. They also lack integration with modern technologies such as sensors, embedded systems, and wireless communication platforms. In contrast, platforms like Arduino have enabled the development of low-cost, sensor-based automation systems, making smart irrigation more accessible to farmers [8][9][10][11].

In conclusion, the rapid advancement of technology has significantly transformed agriculture from a manual, labor-intensive practice into a highly automated and efficient system. Modern irrigation techniques, powered by sensors, communication networks, and renewable energy, have greatly improved water management and crop productivity. However, traditional irrigation methods still suffer from limitations such as lack of real-time monitoring, inefficiency, and high labor dependency. These challenges highlight the importance of adopting advanced technologies to ensure sustainable and efficient agricultural practices for the future.

2.2 Conventional Irrigation Systems

Conventional irrigation systems have been widely used in agriculture for many decades and form the foundation of traditional farming practices. These systems primarily include methods such as flood irrigation, furrow irrigation, and manual watering, where water is supplied to crops based on fixed schedules or the farmer's judgment. While these methods have supported agricultural production for a long time, they are increasingly considered inefficient and inadequate in meeting the demands of modern agriculture, particularly in the context of water scarcity, climate variability, and the need for higher productivity.

One of the key characteristics of conventional irrigation systems is their dependence on manual operation. Farmers typically decide when and how much to irrigate based on visual observation or experience rather than precise data. This lack of scientific

decision-making often results in inconsistent irrigation practices. Early developments in irrigation control, such as microcomputer-based systems, attempted to introduce some level of automation; however, these systems were limited in functionality and lacked real-time data integration [6]. As a result, traditional irrigation systems continue to rely heavily on human intervention, making them labor-intensive and prone to errors.

Another major limitation of conventional irrigation systems is inefficient water usage. Methods like flood irrigation distribute water uniformly across the field without considering the specific needs of different crops or soil conditions. This leads to significant water losses due to evaporation, runoff, and deep percolation. In regions facing water scarcity, such inefficiencies can have serious consequences. Studies on site-specific irrigation management highlight the importance of precise water application, which is not achievable with conventional methods [5]. The inability to control and monitor water usage effectively makes traditional systems unsustainable in the long term.

Conventional irrigation systems also lack real-time monitoring capabilities. Parameters such as soil moisture, temperature, and humidity are not measured continuously, and irrigation decisions are not adjusted based on changing environmental conditions. This results in either over-irrigation or under-irrigation, both of which negatively affect crop growth and yield. In contrast, modern systems using wireless sensor networks provide continuous feedback and enable data-driven irrigation decisions, significantly improving efficiency [1][3]. The absence of such technologies in conventional systems limits their effectiveness.

In addition to inefficiency, traditional irrigation systems are highly dependent on labor availability. Farmers must manually operate pumps, open and close channels, and monitor water distribution across the field. This process is time-consuming and physically demanding, especially for large-scale farms. Labor shortages, which are increasingly common in rural areas, further exacerbate this problem. The lack of automation in conventional systems makes it difficult to manage irrigation effectively, particularly when timely intervention is required.

Another drawback of conventional irrigation systems is their inability to adapt to modern technological advancements. These systems do not integrate with

communication technologies, sensors, or intelligent control mechanisms. As a result, remote monitoring and control are not possible, and farmers must be physically present to manage irrigation. In contrast, advanced irrigation systems utilize communication technologies such as GPRS and IoT to enable remote access and control, improving convenience and efficiency [1]. Similarly, the use of embedded platforms like Arduino has made it possible to develop low-cost automated systems, which are not supported by traditional methods [8][9][10][11].

Table 2.1: Limitations of Conventional Irrigation Systems and Their Impact on Agricultural Efficiency

S.No	Parameter	Conventional Irrigation Systems	Impact on Agriculture	Reference
1	Water Usage Efficiency	High water loss due to evaporation, runoff, percolation	Wastage of water, reduced sustainability	[5]
2	Monitoring Capability	No real-time monitoring of soil/environment	Over-irrigation or under-irrigation	[1][3]
3	Control Mechanism	Manual operation based on farmer experience	Inconsistent irrigation decisions	[6]
4	Labor Dependency	Requires continuous human involvement	Increased labor cost and effort	[6]
5	Adaptability	Cannot adjust to changing weather/soil conditions	Reduced crop productivity	[5]
6	Technology Integration	No sensors, IoT, or communication systems	No remote monitoring/control	[1][8][9]
7	Energy Efficiency	Continuous pump usage without optimization	High energy consumption	[2]
8	Precision Irrigation	Uniform water distribution regardless of crop needs	Inefficient water usage, poor yield	[4]

Energy inefficiency is also a concern in conventional irrigation systems. Pumps are often operated continuously without optimization, leading to excessive energy consumption. With the increasing cost of electricity and limited availability of power in rural areas, this becomes a significant challenge. Modern solutions, such as solar-powered irrigation systems, address this issue by providing sustainable and energy-efficient alternatives [2]. However, such innovations are absent in conventional systems.

Furthermore, traditional irrigation methods do not support precision agriculture. Advanced techniques like automated drip irrigation and optimization-based water allocation allow precise delivery of water based on crop requirements, significantly improving yield and resource utilization [4]. Conventional systems, on the other hand, apply water uniformly without considering variations in soil and crop conditions, leading to suboptimal performance.

In conclusion, while conventional irrigation systems have played a crucial role in agricultural development, they suffer from several limitations, including inefficient water usage, lack of real-time monitoring, high labor dependency, and inability to integrate with modern technologies. These challenges make them unsuitable for addressing the needs of contemporary agriculture. The transition toward automated and intelligent irrigation systems is essential to overcome these limitations and ensure sustainable and efficient farming practices.

2.3 Microcontroller-Based Irrigation Systems

Microcontroller-based irrigation systems represent an important step in the evolution of agricultural automation, bridging the gap between conventional manual irrigation methods and modern smart irrigation technologies. These systems utilize embedded platforms such as microcontrollers to automate irrigation processes based on predefined conditions or sensor inputs. By integrating sensors, control units, and actuators, microcontroller-based systems provide a cost-effective and efficient solution for improving water management in agriculture.

At the core of these systems is the microcontroller, which acts as the central processing unit responsible for acquiring data from sensors, processing it, and generating appropriate control signals. Sensors such as soil moisture sensors, temperature sensors,

and humidity sensors continuously monitor environmental conditions and provide real-time data to the microcontroller. Based on this data, the system determines whether irrigation is required and accordingly controls devices such as water pumps and valves. This automation reduces the dependency on manual labor and ensures more consistent irrigation compared to traditional methods.



Fig 2.1: Soil Moisture Monitoring System with Sensor and Display Interface

Early developments in irrigation automation involved microcomputer-based control systems, which laid the foundation for modern microcontroller-based solutions. Zazueta and Smajstrla discussed the use of microcomputer-based systems for irrigation control, highlighting their ability to automate basic operations such as scheduling and water distribution [6]. Although these early systems were limited in processing power and flexibility, they demonstrated the potential of automation in improving irrigation efficiency.

With advancements in embedded systems, platforms such as Arduino have become widely popular for developing irrigation automation systems due to their simplicity, low cost, and ease of programming. Arduino-based irrigation systems can be easily interfaced with sensors and actuators, making them suitable for both small-scale and large-scale agricultural applications. According to Leo Louis, Arduino serves as an effective tool for study and research in automation systems, enabling rapid prototyping and implementation [8]. Official Arduino resources further emphasize its versatility and accessibility, which have contributed to its widespread adoption in agricultural automation [9][10][11].

Microcontroller-based irrigation systems also support the integration of communication technologies, allowing for remote monitoring and control. For example, systems incorporating wireless communication modules such as GPRS enable farmers to monitor field conditions and control irrigation from remote locations. Research by Gutiérrez et al. demonstrates how communication-enabled irrigation systems can significantly improve efficiency by providing real-time data and control capabilities [1][3]. This integration enhances the functionality of microcontroller-based systems, making them more responsive and user-friendly.

Another important feature of microcontroller-based irrigation systems is their ability to implement intelligent control strategies. By using algorithms and predefined thresholds, these systems can optimize water usage and improve crop yield. For instance, soil moisture-based irrigation ensures that water is supplied only when required, preventing wastage and promoting efficient resource utilization. Advanced approaches such as optimization-based irrigation, including linear programming techniques, further enhance system performance by allocating water resources more effectively [4].

Energy efficiency is also addressed in microcontroller-based systems through the integration of renewable energy sources such as solar power. Solar-powered irrigation systems use microcontrollers to manage energy usage and control irrigation operations, reducing dependence on conventional power sources. The work by JiaUddin et al. highlights the effectiveness of combining solar energy with automated irrigation systems to achieve sustainable and cost-effective solutions [2].

Despite their advantages, microcontroller-based irrigation systems have certain limitations. One of the primary drawbacks is their limited processing capability compared to more advanced platforms like FPGAs. Microcontrollers execute instructions sequentially, which can lead to delays when handling multiple sensor inputs or complex control algorithms. This limitation affects the system's ability to perform real-time processing in large-scale or high-speed applications. Additionally, scalability can be a challenge, as expanding the system to accommodate more sensors or functionalities may require additional hardware or redesign.

Furthermore, while microcontroller-based systems improve automation, they may still lack the robustness and reliability required for critical applications. Software-based

control systems are more prone to issues such as program errors or system crashes, which can disrupt irrigation operations. In contrast, hardware-based solutions like FPGAs offer deterministic performance and higher reliability, making them more suitable for advanced automation systems.

In conclusion, microcontroller-based irrigation systems play a crucial role in the transition from traditional to smart agriculture. They provide a cost-effective and accessible solution for automating irrigation, improving water efficiency, and reducing labor dependency. By integrating sensors, communication technologies, and renewable energy sources, these systems have significantly enhanced agricultural practices. However, their limitations in processing speed, scalability, and reliability highlight the need for more advanced technologies in certain applications. Nevertheless, microcontroller-based systems remain an essential component of modern irrigation solutions and continue to contribute to the advancement of sustainable agriculture.

2.4 IoT-Based Smart Irrigation Systems

IoT-based smart irrigation systems represent a major advancement in modern agriculture by integrating Internet of Things (IoT) technologies with traditional irrigation practices. These systems utilize interconnected sensors, communication networks, and intelligent control units to monitor environmental conditions and automate irrigation processes in real time. By enabling data-driven decision-making, IoT-based irrigation systems significantly improve water efficiency, reduce labor requirements, and enhance crop productivity.

At the core of IoT-based smart irrigation systems is a network of sensors deployed across the agricultural field. These sensors continuously measure key parameters such as soil moisture, temperature, humidity, and water levels. The collected data is transmitted through communication technologies such as wireless sensor networks (WSNs), Wi-Fi, or GPRS to a central controller or cloud platform. Research by Gutiérrez et al. demonstrates how wireless sensor networks combined with GPRS modules enable efficient data collection and remote monitoring in irrigation systems [1][3]. This real-time data acquisition forms the basis for intelligent irrigation decisions.

One of the most significant features of IoT-based systems is remote monitoring and

control. Farmers can access real-time information about their fields through mobile applications or web interfaces, allowing them to monitor soil conditions and control irrigation systems from anywhere. This eliminates the need for constant physical presence in the field and enhances convenience. The integration of communication technologies in irrigation systems has proven to be highly effective in improving responsiveness and system efficiency [1].

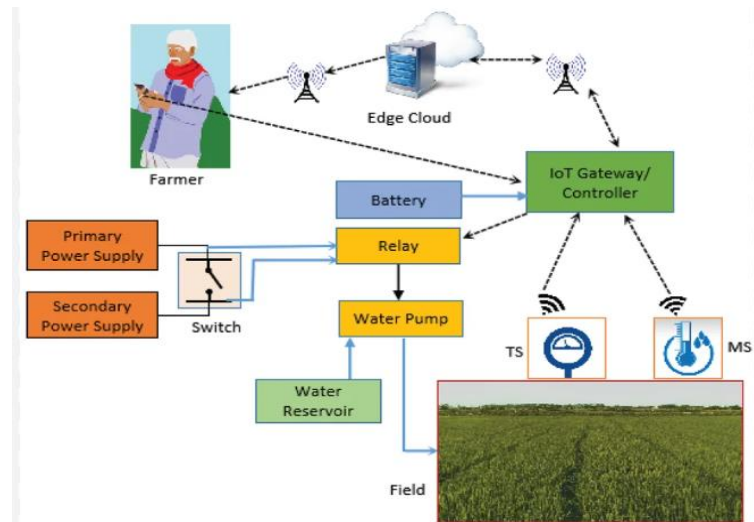


Fig 2.2: IoT-Based Smart Irrigation System Architecture with Cloud and Remote Monitoring

Automation is another key aspect of IoT-based smart irrigation. Based on sensor data, the system can automatically determine when irrigation is required and control pumps or valves accordingly. This ensures that water is supplied only when necessary, preventing over-irrigation and under-irrigation. Early irrigation control systems relied on microcomputers for automation, but modern IoT systems offer significantly enhanced capabilities due to improved connectivity and processing power [6]. Autonomous irrigation controllers capable of site-specific management further enhance precision and efficiency [5].

Energy efficiency and sustainability are also important considerations in IoT-based irrigation systems. Many modern systems integrate renewable energy sources such as solar power to operate sensors and control units. The study by JiaUddin et al. highlights the effectiveness of solar-powered irrigation systems in reducing energy costs and ensuring reliable operation in remote areas [2]. Combining IoT with renewable energy results in environmentally sustainable and cost-effective irrigation solutions.

IoT-based irrigation systems also support advanced data analysis and optimization techniques. By analyzing historical and real-time data, these systems can optimize irrigation schedules and water distribution. Techniques such as intelligent drip irrigation and optimization-based water allocation improve water usage efficiency and crop yield. Giri and Wavhal proposed an automated drip irrigation system using linear programming, which demonstrates the potential of combining IoT with optimization methods for better resource management [4].

Another important advantage of IoT-based systems is scalability and flexibility. These systems can be easily expanded by adding more sensors or integrating additional functionalities such as weather forecasting and crop monitoring. Embedded platforms like Arduino play a crucial role in implementing IoT-based irrigation systems due to their ease of use and compatibility with various sensors and communication modules [8][9][10][11]. This makes IoT solutions accessible to a wide range of users, from small-scale farmers to large agricultural enterprises.

In addition to irrigation, IoT technologies enable integration with other smart farming applications. For example, machine vision systems for crop monitoring and automated machinery can work alongside IoT-based irrigation to create a comprehensive smart agriculture ecosystem. Vision-based systems for agricultural machinery, as discussed by Astrand and Baerveldt, demonstrate how advanced technologies can enhance precision and efficiency in farming operations [7].

Despite their numerous advantages, IoT-based smart irrigation systems also face certain challenges. These include initial installation costs, network connectivity issues in rural areas, and the need for technical expertise to manage and maintain the system. However, ongoing advancements in technology and decreasing costs of sensors and communication devices are making these systems increasingly affordable and practical.

In conclusion, IoT-based smart irrigation systems have transformed traditional irrigation practices by introducing real-time monitoring, automation, and remote control. By leveraging sensor networks, communication technologies, and data analytics, these systems provide efficient and sustainable solutions for water management in agriculture. The research and developments presented in the referenced

works highlight the significant potential of IoT in improving agricultural productivity and addressing the challenges of modern farming.

2.5 FPGA-Based Control Systems

FPGA-based control systems have gained significant attention in recent years due to their ability to provide high-speed, reliable, and flexible solutions for automation applications. In the domain of agriculture, particularly in smart irrigation systems, FPGAs (Field Programmable Gate Arrays) offer a powerful alternative to conventional microcontroller-based systems. By enabling parallel processing, real-time control, and hardware-level implementation of logic, FPGA-based systems enhance the efficiency and performance of automated irrigation and other agricultural processes.

At the core of FPGA-based control systems is the concept of programmable hardware. Unlike microcontrollers, which execute instructions sequentially through software, FPGAs allow designers to implement custom hardware logic using hardware description languages such as Verilog or VHDL. This enables multiple operations to be executed simultaneously, making FPGAs highly suitable for applications that require real-time processing of multiple inputs. In irrigation systems, where parameters such as soil moisture, temperature, humidity, and water levels must be continuously monitored, FPGA-based systems can process all sensor data in parallel, ensuring faster and more accurate decision-making. The need for real-time data acquisition and control has been emphasized in advanced irrigation systems using wireless sensor networks [1][3].

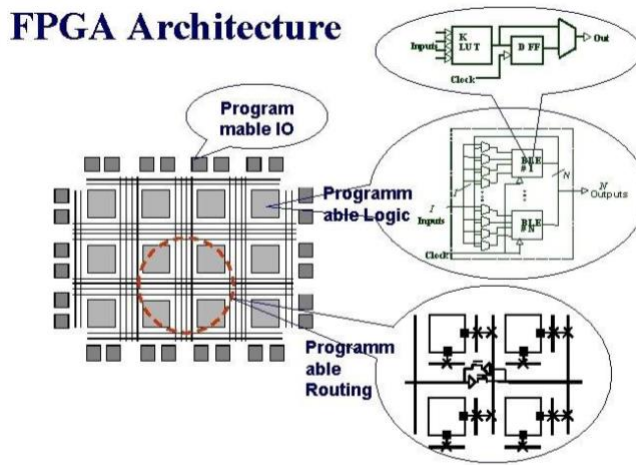


Fig 2.3: Internal Architecture of FPGA Showing Programmable Logic Blocks, Routing, and I/O Units

One of the major advantages of FPGA-based control systems is their ability to provide deterministic and reliable performance. Since the control logic is implemented directly in hardware, the system operates with precise timing and minimal latency. This is particularly important in irrigation applications, where immediate response to changing environmental conditions is required. Earlier automation systems based on microcomputers provided basic control functionality but were limited in speed and responsiveness [6]. FPGA-based systems overcome these limitations by offering faster execution and predictable behavior, making them more suitable for critical control applications.

Flexibility and reconfigurability are also key features of FPGA-based systems. Designers can modify the functionality of the system by updating the hardware description code without changing the physical hardware. This allows the system to be adapted for different crops, soil conditions, or irrigation strategies. For example, advanced irrigation techniques such as intelligent drip irrigation and optimization-based water allocation require customizable control logic, which can be efficiently implemented using FPGAs [4]. This adaptability makes FPGA-based systems highly versatile for a wide range of agricultural applications.

Another important aspect of FPGA-based control systems is their ability to integrate with various technologies, including sensors, communication modules, and energy systems. Modern irrigation systems often rely on wireless communication technologies such as GPRS or IoT platforms for remote monitoring and control. FPGA-based systems can interface with these communication modules, enabling seamless data transfer and system coordination. Research on wireless sensor network-based irrigation systems highlights the importance of such integration for improving efficiency and enabling remote operation [1]. Additionally, FPGA systems can be combined with renewable energy sources such as solar power to create sustainable irrigation solutions, as demonstrated in solar-powered irrigation systems [2].

FPGAs also support advanced applications such as machine vision and intelligent automation. In agriculture, vision-based systems are used for tasks like crop monitoring and row-following in autonomous machinery. These applications require high-speed image processing, which can be efficiently handled by FPGA-based systems due to their

parallel processing capabilities. The work by Astrand and Baerveldt on vision-based agricultural machinery demonstrates the potential of integrating advanced technologies into farming systems [7]. By combining such technologies with FPGA-based control, it is possible to develop highly sophisticated and fully automated agricultural systems.

Compared to microcontroller-based systems, FPGA-based control systems offer superior performance in terms of speed, scalability, and reliability. While platforms like Arduino are widely used for low-cost and small-scale applications due to their simplicity and ease of use [8][9][10][11], they are limited by sequential processing and lower computational power. FPGAs, on the other hand, can handle complex algorithms, multiple sensor inputs, and high-speed operations, making them suitable for large-scale and high-performance applications.

Despite their advantages, FPGA-based systems also present certain challenges. These include higher initial cost, increased design complexity, and the need for specialized knowledge in hardware description languages. However, with the development of advanced FPGA platforms such as system-on-chip (SoC) devices that combine programmable logic with embedded processors, these challenges are being gradually reduced. Such platforms provide a balance between hardware acceleration and software flexibility, making FPGA-based solutions more accessible to researchers and developers.

In conclusion, FPGA-based control systems play a crucial role in advancing automation in agriculture by providing high-speed, reliable, and flexible solutions. Their ability to process data in parallel, implement real-time control, and integrate with modern technologies makes them highly suitable for smart irrigation systems and other agricultural applications. By overcoming the limitations of traditional and microcontroller-based systems, FPGA-based control systems contribute to improved efficiency, resource management, and sustainability in modern agriculture.

2.6 Comparative Analysis of Existing Systems

A comparison of different irrigation systems highlights the strengths and weaknesses of each approach:

- Conventional systems are simple but lack automation and efficiency

- Microcontroller-based systems provide basic automation but are limited by processing speed and scalability
- IoT-based systems enable remote monitoring but suffer from latency and connectivity issues
- FPGA-based systems offer high-speed processing, parallelism, and reliable real-time control

From this analysis, it is evident that FPGA-based systems provide a more robust and efficient solution for automated irrigation, particularly in applications requiring real-time responsiveness and high reliability.

Table 2.2: Comparative Analysis of Irrigation Systems Based on Automation, Speed, Real-Time Capability, Cost, and Reliability

S.No	System Type	Key Features	Advantages	Limitations	Reference
1	Conventional Irrigation	Manual control, fixed schedules	Simple, low initial cost	No automation, water wastage, labor-intensive	[5][6]
2	Microcontroller-Based	Sensor-based control using embedded systems (e.g., Arduino)	Low cost, basic automation, easy implementation	Limited processing speed, sequential execution, less scalable	[6][8][9]
3	IoT-Based Irrigation	Wireless sensors, remote monitoring, cloud connectivity	Remote access, real-time data monitoring	Network dependency, latency, connectivity issues	[1][3]
4	FPGA-Based Irrigation	Parallel processing, hardware-level control	High speed, real-time response, high reliability	Higher cost, design complexity	[4]

2.7 Research Gap

Despite significant advancements in irrigation technologies, there exists a gap in the implementation of high-speed, real-time control systems using FPGA technology. Most existing systems rely on microcontrollers or cloud-based processing, which may not meet the requirements of real-time operation and scalability.

There is a need for a system that:

- Processes sensor data in real-time

- Provides immediate control actions
- Operates independently of network connectivity
- Ensures high reliability and performance

This chapter reviewed various irrigation systems, including conventional, microcontroller-based, IoT-based, and FPGA-based approaches. While each method has its advantages, they also exhibit certain limitations that affect their efficiency and reliability.

The analysis clearly indicates that FPGA-based systems offer superior performance in terms of speed, parallel processing, and real-time decision-making. Therefore, the proposed FPGA-based automated irrigation system aims to address the limitations of existing approaches and provide an efficient and reliable solution for smart agriculture.

Chapter-3: PROPOSED SYSTEM DESIGN

3.1 Introduction

The system design forms the backbone of the proposed FPGA-Based Automated Water Management System for Smart Agriculture. It defines how different hardware and software components interact to achieve efficient irrigation control. The primary objective of the system design is to ensure real-time monitoring of environmental parameters and automatic decision-making for irrigation.

Modern agricultural systems require precision, reliability, and efficiency in water management. Traditional irrigation systems lack adaptability and real-time responsiveness, which often leads to inefficient water usage. To overcome these limitations, the proposed system integrates sensing devices, embedded processing units, and communication modules to create an intelligent irrigation framework.

The system is designed using a modular approach, where each component performs a specific function such as sensing, data conversion, processing, communication, and actuation. This modularity enhances system flexibility, making it easier to upgrade or extend in the future.

A key highlight of the proposed design is the use of FPGA-based processing, which enables high-speed parallel execution of control logic. Unlike conventional systems, this approach ensures minimal delay in decision-making, making the system suitable for real-time applications.

Overall, the system design focuses on:

- Accurate sensing of soil and water conditions
- Fast and reliable data processing
- Automated control of irrigation
- Reduction of water wastage and human effort

3.2 Overall System Architecture

The overall system architecture consists of multiple interconnected components that work together to achieve automated irrigation. The system is designed as a closed-loop control system where continuous feedback from sensors is used to control the irrigation process.

The major components of the system include:

- Soil moisture sensor
- Water level sensor
- Arduino Uno (ADC unit)
- Zynq-7000 SoC (FPGA processing unit)
- LoRa communication modules
- Relay module
- Water pump (motor)

The sensors are responsible for collecting real-time environmental data. Since these sensors produce analog signals, the Arduino Uno is used to convert them into digital signals. The converted data is then transmitted wirelessly using LoRa modules.

The FPGA receives the sensor data and processes it using predefined logic implemented in Verilog HDL. Based on the processed data, control signals are generated to operate the motor through a relay module.

The closed-loop architecture ensures continuous monitoring and automatic response to changes in soil conditions. This improves system efficiency and ensures optimal water usage.

Table 3.1: Components of Proposed FPGA-Based Smart Irrigation System

S.No	Component	Function	Role in System	Reference
1	Soil Moisture Sensor	Measures soil water content	Provides input for irrigation decision	[1][3]
2	Water Level Sensor	Detects water availability in tank/source	Prevents dry running of pump	[1]
3	Arduino Uno (ADC Unit)	Converts analog sensor signals to digital data	Acts as interface between sensors and FPGA	[8][9]
4	Zynq-7000 SoC (FPGA Unit)	Processes data using Verilog-based logic	Core decision-making and control unit	[4]
5	LoRa Communication Modules	Enables long-range wireless data transmission	Supports remote monitoring and communication	[1][3]
6	Relay Module	Acts as an electronic switch	Controls motor ON/OFF operation	[6]
7	Water Pump (Motor)	Pumps water to the field	Executes irrigation process	[2]

3.3 Block Diagram of the System

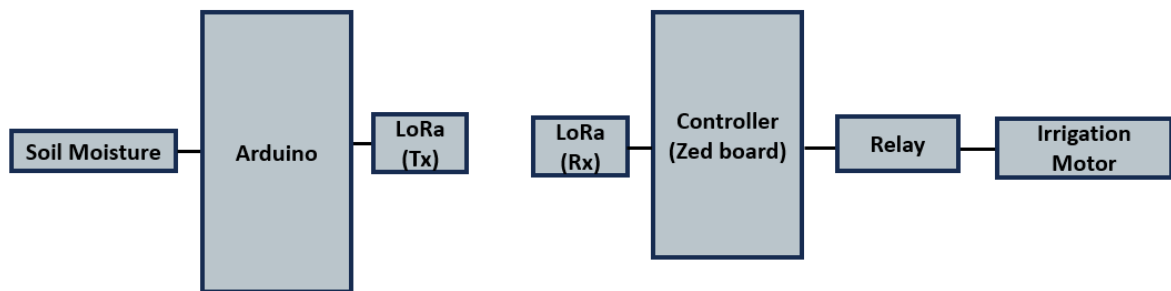


Fig 3.1: Block Diagram of FPGA-Based Automated Irrigation System Using LoRa Communication

The block diagram provides a graphical representation of the system architecture and shows how different components are interconnected. It illustrates the flow of data from sensors to the processing unit and finally to the actuator.

The system begins with sensors that collect real-time data from the field. The analog

signals generated by the sensors are fed into the Arduino Uno for analog-to-digital conversion. The digital data is then transmitted via a LoRa transmitter module.

At the receiver side, the LoRa module connected to the FPGA receives the transmitted data. The FPGA processes this data using control logic and generates output signals accordingly. These signals are then used to control the relay, which switches the motor ON or OFF.

The block diagram clearly represents the closed-loop control mechanism of the system, where sensor feedback continuously influences the operation of the motor.

3.4 Description of System Components

Soil Moisture Sensor- The soil moisture sensor is used to measure the water content present in the soil. It provides an analog output voltage proportional to the moisture level. This sensor plays a critical role in determining whether irrigation is required.

Arduino-Based ADC Unit- The sensor outputs are analog in nature and cannot be directly processed by digital systems. Therefore, an Arduino Uno is used to perform Analog-to-Digital Conversion (ADC). The Arduino reads analog values from the sensors and converts them into digital signals, which are then transmitted to the FPGA.

FPGA Control Unit- The Zynq-7000 SoC acts as the central processing unit of the system. The FPGA executes control logic written in Verilog HDL to analyze sensor inputs and make real-time decisions.

Key functions of FPGA:

- Parallel processing of sensor data
- High-speed decision-making
- Reliable and deterministic operation

Motor Control Unit- The motor is used to pump water for irrigation. Based on the control signals generated by the FPGA, the motor is switched ON or OFF. A driver circuit or relay is typically used to interface the FPGA output with the motor.

3.5 Working Principle of the System

The system operates in a sequential yet real-time manner as described below:

1. The soil moisture sensor continuously measures the moisture level of the soil.
2. The analog signal from the soil moisture sensor is sent to the Arduino Uno for ADC conversion.
3. The processed digital sensor data is transmitted wirelessly using a LoRa transmitter module connected to the Arduino Uno.
4. A LoRa receiver module interfaced with the Zynq-7000 SoC receives the transmitted sensor data.
5. The FPGA processes the received input using predefined threshold logic implemented in Verilog HDL.
6. If the soil moisture is below the threshold, the FPGA generates a signal to turn ON the motor.
7. When the desired moisture level is reached, the FPGA turns OFF the motor automatically.

This process continues in a loop, ensuring continuous monitoring and control.

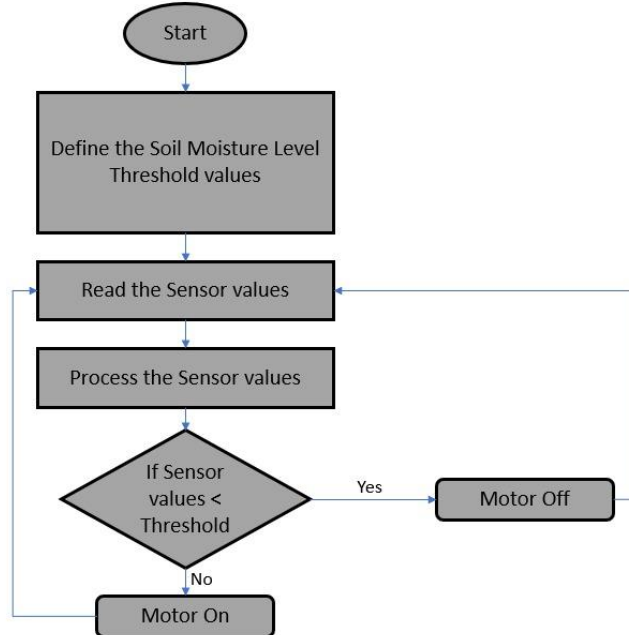


Fig 3.6: Flowchart of FPGA-Based Automated Irrigation Control System

3.6 Design Considerations

The design of the system is based on several critical factors to ensure efficient and reliable

operation.

Real-Time Response:

The system must respond immediately to changes in soil moisture. FPGA-based processing ensures minimal delay.

Reliability:

The system should avoid false triggering due to noise or fluctuations in sensor readings. Proper filtering and threshold selection are required.

Scalability:

The system should be capable of supporting additional sensors and functionalities in the future.

Power Efficiency:

The system should minimize power consumption, especially for remote agricultural areas.

Safety:

The motor should not operate when there is no water available. The water level sensor ensures safe operation.

Chapter- 4: HARDWARE DESIGN

4.1 Introduction

The hardware design of the proposed FPGA-Based Automated Water Management System plays a crucial role in ensuring reliable and efficient operation. It involves the selection, integration, and interfacing of various electronic components required to implement real-time irrigation control.

Modern smart irrigation systems require seamless interaction between sensing units, processing modules, and actuation systems. The hardware design ensures that these components work together in a synchronized manner to achieve accurate monitoring and control of irrigation processes.

The proposed system integrates sensors to collect environmental data, an Arduino-based interface for signal conditioning and analog-to-digital conversion, an FPGA-based processing unit for real-time decision-making, and a motor control system for irrigation.

Special attention is given to:

- Signal compatibility between components
- Reliable communication interfaces
- Power management and safety
- Real-time responsiveness

The hardware architecture is designed in a modular fashion, allowing easy expansion and maintenance. Each component is carefully selected based on performance, cost, and compatibility with the overall system.

4.2 Overview of Hardware Components

The proposed system consists of several hardware components, each performing a specific function. These components work together to form a complete automated irrigation system.

The major components include:

- Soil Moisture Sensor
- Arduino Uno
- Zynq-7000 SoC (FPGA)
- Relay Driver Circuit
- Water Pump (Motor)
- Power Supply Unit

Each component is selected based on its ability to operate reliably under real-time conditions and its compatibility with other system components.

The soil moisture sensor collects data from the field, while the Arduino converts analog signals into digital form. The FPGA processes this data and generates control signals. The relay acts as an interface between low-power FPGA outputs and high-power motor input.

Table 4.1: Hardware Components Specification and Their Functional Description

S.No	Component	Specification / Type	Function	Reference
1	Soil Moisture Sensor	Resistive/Capacitive Soil Sensor	Measures soil moisture level in the field	[1][3]
2	Arduino Uno	ATmega328P Microcontroller Board	Converts analog sensor signals to digital data (ADC operation)	[8][9][10]
3	Zynq-7000 SoC (FPGA)	Xilinx FPGA with ARM Cortex-A9	Processes data and generates control signals using Verilog HDL	[4]
4	Relay Driver Circuit	Electromechanical / Transistor-Based Relay	Interfaces low-power FPGA output with high-power motor	[6]
5	Water Pump (Motor)	DC/AC Motor	Pumps water for irrigation	[2]
6	Power Supply Unit	Regulated DC Supply (5V/12V)	Provides required power to all system components	[2]

4.3 Soil Moisture Sensor

Description- The soil moisture sensor is used to measure the volumetric water content present in the soil. It typically consists of two conductive probes that are inserted into the soil. The resistance between the probes varies depending on the moisture level.

Working Principle

- Wet soil → Low resistance → Higher conductivity
- Dry soil → High resistance → Lower conductivity



Fig 4.1: Soil Moisture Sensor Module Used for Measuring Soil Water Content

The sensor outputs an **analog voltage** proportional to the moisture level, which is later converted into digital form.

Role in System

- Determines irrigation requirement
- Provides continuous feedback to the control unit

4.4 Arduino-Based ADC Interface

Need for ADC

Since sensor outputs are analog signals, they cannot be directly processed by digital systems like FPGA. Therefore, an ADC unit is required.

Implementation Using Arduino

The Arduino Uno is used as an ADC interface due to its built-in analog input channels and ease of programming.

Features

- 10-bit ADC resolution (0–1023)
- Multiple analog input pins
- Serial communication support

Function in System

- Reads analog values from sensors
- Converts them into digital values
- Sends processed data to FPGA



Fig 4.2: Arduino Uno with ADC input channels

4.5 FPGA Development Board

Description

The Zynq-7000 SoC is used as the central processing unit. It combines programmable logic (FPGA) with processing capabilities, making it suitable for real-time embedded applications.

Key Features

- High-speed parallel processing
- Configurable logic blocks
- Low latency operation
- Flexible interfacing

Role in System

- Receives digital sensor data
- Executes control logic using Verilog HDL
- Generates control signals for motor operation

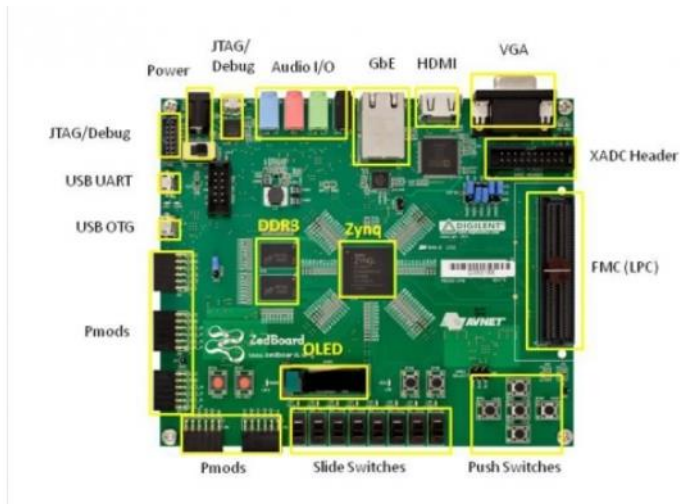


Fig 4.3: ZedBoard (Zynq-7000 SoC) FPGA Development Board with Peripheral Interfaces

Why FPGA Over Microcontroller

- FPGA supports **parallel execution**, unlike sequential microcontrollers
- Faster response time
- Better for real-time control applications
- Higher reliability

4.6 Motor and Relay Driver Circuit

Motor Description

A water pump motor is used to supply water for irrigation. It requires higher voltage and current than what FPGA can directly provide.

Need for Driver Circuit

The FPGA outputs low-power digital signals, which cannot drive the motor directly. Therefore, a relay driver circuit is used as an interface.

Relay Operation

- FPGA output HIGH → Relay activated → Motor ON
- FPGA output LOW → Relay deactivated → Motor OFF

Safety Considerations

- Electrical isolation between control and power circuits
- Protection against voltage spikes



Fig 4.4: Electrical Control Panel for Motor Control and Power Distribution

4.7 Power Supply Unit

Requirements

Different components require different voltage levels, making power management an important aspect of system design.

- Arduino $\rightarrow$ 5V
- FPGA $\rightarrow$ 3.3V / 5V
- Sensors $\rightarrow$ 3.3V / 5V
- Motor $\rightarrow$ High voltage
- Implementation

A regulated power supply is used to ensure stable voltage levels. Voltage regulators and adapters are used to provide appropriate supply to each component.

Importance

- Prevents damage to components
- Ensures stable operation
- Improves system reliability

4.8 Hardware Integration

The hardware integration involves connecting all components to form a complete working system. Proper interfacing is required to ensure smooth data flow and reliable operation.

The LoRa module is connected to the Arduino using UART communication. The Arduino collects sensor data and transmits it wirelessly to the FPGA.

Key Connections

- Arduino ↔ Sensors (Analog pins)
- Arduino ↔ LoRa (UART: TX/RX)
- LoRa ↔ FPGA (Receiver side)
- FPGA ↔ Relay
- Relay ↔ Motor

Working Flow

- Sensors → Arduino
- Arduino → LoRa transmission
- LoRa → FPGA reception
- FPGA → Relay → Motor

Chapter- 5: SOFTWARE DESIGN

5.1 Introduction

The software design of the proposed system focuses on implementing the control logic required for automated irrigation using Verilog Hardware Description Language (HDL). The software component is responsible for processing digital inputs received from sensors and generating appropriate control signals for motor operation.

The design is implemented on the Zynq-7000 SoC, enabling real-time decision-making through parallel processing. The software is modular in nature, ensuring scalability and ease of debugging.

5.2 Design Methodology

The software design follows a structured approach consisting of:

1. Input acquisition from sensors (via Arduino Uno)
2. Data comparison with predefined threshold values
3. Decision-making using control logic
4. Output generation for motor control

The system continuously monitors sensor inputs and updates the motor status accordingly.

5.3 Verilog Implementation

Module Description

The Verilog code is designed to implement the control logic using combinational logic. The module takes sensor inputs and produces a control output for the motor.

Algorithm: FPGA-Based Automated Irrigation System

Step 1: System Initialization

- Start the system.
- Initialize all components:
- Set motor state = OFF
- Reset FPGA registers and counters
- Initialize communication (LoRa UART)
- Set predefined moisture threshold value.

Step 2: Data Acquisition

- Soil moisture sensor measures analog moisture value.
- Water level sensor checks availability of water.
- Arduino Uno converts analog sensor data → digital form (ADC).
- Transmit digital data to FPGA using LoRa communication.

Step 3: Data Reception (UART – FPGA)

- FPGA receives serial data through UART module.
- Detect start bit and validate incoming data frame.
- Read incoming data bits and store them.
- Verify stop bit:
- If valid → proceed
- Else → generate error and discard data

Step 4: Data Decoding

- Check for valid packet header (e.g., "+RCV=").
- Extract:
- Address field
- Length field
- Payload (sensor data)
- Convert received ASCII data → numerical value.

Step 5: Data Processing (Decision Logic)

- Compare received moisture value with threshold:
- IF (Moisture Value < Threshold) AND (Water Available)
- → Motor = ON
- ELSE
- → Motor = OFF

Step 6: Control Signal Generation

- FPGA generates control signal based on decision.

- Send control signal to relay driver circuit.

Step 7: Actuation

- Relay switches:
- Motor ON → Irrigation starts
- Motor OFF → Irrigation stops

Step 8: Continuous Monitoring (Closed Loop)

- Repeat steps 4–18 continuously.
- Update motor status in real-time based on sensor inputs.

Step 9: Error Handling

- If invalid data or communication error:
- Ignore data
- Maintain previous motor state
- Return to monitoring state.

Step 10: End Condition

- System runs continuously unless reset or powered off.

UART Code:

```
`timescale 1ns/1ps
```

```
module rx_module
(
    input          rx_clk,
    input          res,
    input          rx_in,
    output wire [7:0] rx_data_reg,
    output wire     busy,
    output wire     data_valid,
    output wire     error
);
```

```

parameter idle      = 2'b00;
parameter start_bit = 2'b01;
parameter data_bits = 2'b10;
parameter stop_bit  = 2'b11;

localparam count_width = 14;
localparam clks_per_bit = 10416;

reg      rx_data1 = 1'b1;
reg      rx_data  = 1'b1;
reg [7:0] rx_output_data;
reg      rx_busy;
reg      rx_data_valid;
reg      rx_error;

reg [count_width-1:0] counter = 0;
reg [2:0] bit_index = 3'b000;
reg [1:0] current_state = 2'b00;

always @(posedge rx_clk or negedge res) begin

    if(!res) begin
        rx_data1 <= 1;
        rx_data  <= 1;
    end
    else begin
        rx_data1 <= rx_in;
        rx_data  <= rx_data1;
    end
end

always @(posedge rx_clk or negedge res) begin

    if(!res) begin
        current_state <= idle;

```

```

    rx_output_data <= 8'b0;
    rx_data_valid <= 1'b0;
    rx_error <= 1'b0;
    rx_busy <= 1'b0;
    counter <= 0;
    bit_index <= 3'b000;
end

else begin

    case(current_state)

    idle: begin
        rx_error <= 1'b0;
        rx_data_valid <= 1'b0;
        rx_busy <= 1'b0;
        counter <= 0;
        bit_index <= 3'b0;

        if(rx_data == 1'b0) begin
            current_state <= start_bit;
        end
    end

    start_bit: begin

        if(counter == (clks_per_bit-1)/2) begin

            if(rx_data == 1'b0) begin
                counter <= 0;
                rx_busy <= 1'b1;
                current_state <= data_bits;
            end
        else begin
            current_state <= idle;

```

```

        rx_error <= 1'b1;
    end
end

else begin
    counter <= counter + 1'b1;
end
end

data_bits: begin

    if(counter < clk_per_bit-1) begin
        counter <= counter + 1'b1;
    end

    else begin
        counter <= 0;
        rx_output_data[bit_index] <= rx_data;

        if(bit_index < 7) begin
            bit_index <= bit_index + 1'b1;
        end

        else begin
            bit_index <= 3'b0;
            current_state <= stop_bit;
        end
    end
end

stop_bit: begin

    if(counter < clk_per_bit-1) begin
        counter <= counter + 1'b1;
    end

```

```

else begin

    if(rx_data == 1'b1) begin
        rx_data_valid <= 1'b1;
        counter <= 0;
        rx_busy <= 1'b0;
        current_state <= idle;
    end

    else begin
        rx_error <= 1'b1;
        rx_output_data <= 8'b0;
        rx_data_valid <= 0;
        current_state <= idle;
    end
end

end

default : current_state <= idle;

endcase
end
end

assign rx_data_reg = rx_output_data;
assign busy = rx_busy;
assign data_valid = rx_data_valid;
assign error = rx_error;

```

endmodule

Top Module:

```

module lora_uart (
    input    rx_clk,
    input    res,

```



```

    input    rx_in,
    output   motor_out
);

wire [7:0] rx_data_reg;
wire busy, data_valid, error;

rx_module uut (
    rx_clk, res, rx_in,
    rx_data_reg, busy, data_valid, error
);

localparam IDLE  = 0,
            HEADER = 1,
            ADDR  = 2,
            LENGTH = 3,
            PAYLOAD = 4,
            DONE  = 5;

reg [2:0] state;
reg cr_seen;

parameter THRESHOLD = 10'd600;

reg [2:0] header_index;
reg [7:0] length;
reg [7:0] payload_count;
reg [15:0] number;
reg [9:0] received_value;
reg motor_ctrl;

always @(posedge rx_clk or negedge res) begin
    if (!res) begin
        state <= IDLE;
        header_index <= 0;
        length <= 0;
        payload_count <= 0;
        number <= 0;

```

```

received_value <= 0;
motor_ctrl <= 0;
cr_seen <= 0;
end
else if (data_valid) begin
    case(state)

        IDLE: begin
            if (rx_data_reg == 8'h2B) begin
                state <= HEADER;
                header_index <= 0;
            end
        end

        HEADER: begin
            case(header_index)
                0: if (rx_data_reg == "R") header_index <= 1; else state <= IDLE;
                1: if (rx_data_reg == "C") header_index <= 2; else state <= IDLE;
                2: if (rx_data_reg == "V") header_index <= 3; else state <= IDLE;
                3: if (rx_data_reg == "=") state <= ADDR; else state <= IDLE;
            endcase
        end

        ADDR: begin
            if (rx_data_reg == 8'h2C) begin
                state <= LENGTH;
                length <= 0;
            end
        end

        LENGTH: begin
            if (rx_data_reg == 8'h2C) begin
                state <= PAYLOAD;
                payload_count <= 0;
                number <= 0;
            end
            else if (rx_data_reg >= 8'h30 && rx_data_reg <= 8'h39) begin
                length <= (length * 10) + (rx_data_reg - 8'h30);
            end
        end
    endcase
end

```

```

        end
    end

    PAYLOAD: begin
        if (payload_count < length) begin
            if (rx_data_reg >= 8'h30 && rx_data_reg <= 8'h39) begin
                number <= (number * 10) + (rx_data_reg - 8'h30);
            end
            payload_count <= payload_count + 1;
            if (payload_count == length-1) begin
                state <= DONE;
            end
        end
    end

    DONE: begin
        received_value <= number[9:0];
        if (number < THRESHOLD)
            motor_ctrl <= 0;
        else
            motor_ctrl <= 1;

            if (rx_data_reg == 8'h0D) begin
                cr_seen <= 1;
            end
            else if (rx_data_reg == 8'h0A && cr_seen) begin
                cr_seen <= 0;
                state <= IDLE;
            end
            else begin
                cr_seen <= 0;
            end
        end
    end

endcase

end

end

assign motor_out = motor_ctrl;

```

endmodule

5.5 Procedure for software implementation

Step 1: Launch Vivado and Start New Project:

- Open Vivado 2018.2 from the Start menu.
- Click Next.

Step 2: Enter Project Name and Location:

- Enter a suitable Project Name
- Choose a Project Location where the project files will be stored.
- Keep the option Create project subdirectory checked.
- Click Next.

Step 3: Select Project Type (RTL Project):

- Select RTL Project as the project type.
- (Optional) If you plan to add Verilog files later, keep Do not specify sources at this time checked.
- Click Next.

Step 4: Select Target Board (Zynq 7000 SoC):

- In the Default Part window of the New Project Wizard, switch to the Boards tab.
- In the search box, type Zedboard.
- Select Zynq 7000 Soc (Zedboard) from the list.
- Click Next to proceed and then Finish to create the project.

Step 5: Add Design Sources:

- In the Flow Navigator, click Add Sources.
- Select Add or create design sources and click Next.

Step 6: Create Verilog Module (Define I/O Ports)

- In the Add Sources flow, choose Create File and select Verilog as the file type.
- Enter a file/module name (e.g., p1) and proceed.
- The Define Module dialog box appears.
- Enter the Module name.
- Add I/O ports using the '+' (Add Port) button:
- Define input ports
- Define output ports
- Set the Direction for each port correctly (Input/Output).

- Click OK to generate the Verilog module template.
- Click Add Sources again from the Project Manager.
- Select Add or create simulation sources and click Next.

Step 7: Write RTL Code in the Verilog Source File:

- In the Sources panel, locate the newly created Verilog file (e.g., p1) under Design Sources.
- Double-click the file to open it in the Vivado editor.
- Write the required RTL logic inside the module
- Save the file after writing the code

Step 8: Add Simulation Sources

- In the editor window, write the required testbench code to apply inputs and observe outputs of the module.
- Save the file using Ctrl + S or by clicking the Save icon.
- Verify that the testbench file is listed under Simulation Sources.

Step 9: Define Testbench Module and Ports:

- When the Define Module dialog opens for the simulation file, enter a suitable module name
- For testbenches, input/output ports are typically not required (stimulus is driven internally).
- Click OK to generate the testbench template.

Step 10: Write and Save Testbench Code :

Step 11: Run Behavioral (Functional) Simulation

- In the Flow Navigator, under Simulation, click Run Simulation → Run Behavioral Simulation.
- Vivado launches the simulator (XSIM) and loads the testbench.
- Wait for the simulation to initialize.

Step 12: Observe Simulation Waveforms

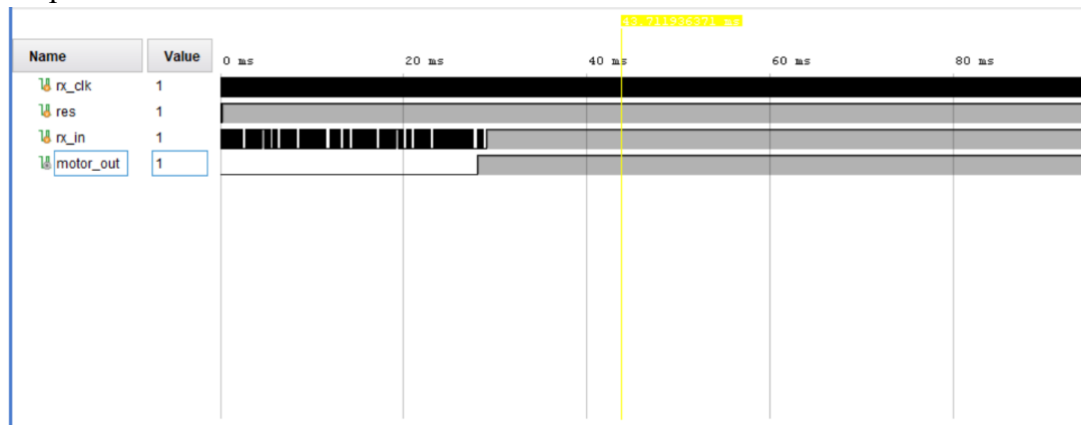


Fig 5.1: Simulation Waveform Showing Motor Control Output Based on Input Signal Conditions

- In the simulation window, observe the waveform of the tx signal.
- Use the time cursor and zoom controls to inspect signal transitions.
- Verify that each UART frame correctly represents the ASCII values of the characters in the string "Hello, World".
- Confirm the baud rate timing by measuring the duration of each bit period and comparing it to the expected value ($1/9600$ seconds $\approx 104 \mu\text{s}$).

Step 13: Launch Synthesis Run:

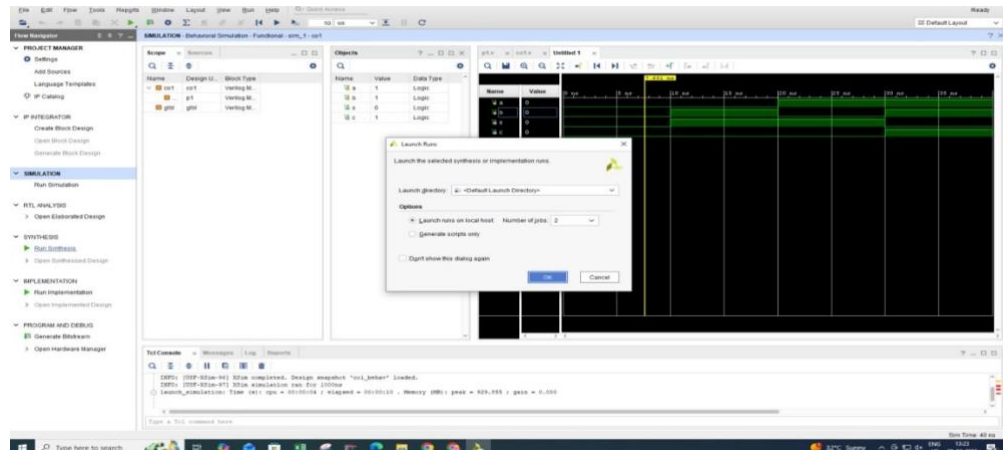


Fig 5.2: Interface of Synthesis Run Launch in Xilinx Vivado

- In the Flow Navigator, click Run Synthesis.
- When the Launch Runs dialog appears, keep the default settings (local host, default number of jobs).
- Click OK to start synthesis.

Step 14: Complete Synthesis and Proceed to Implementation

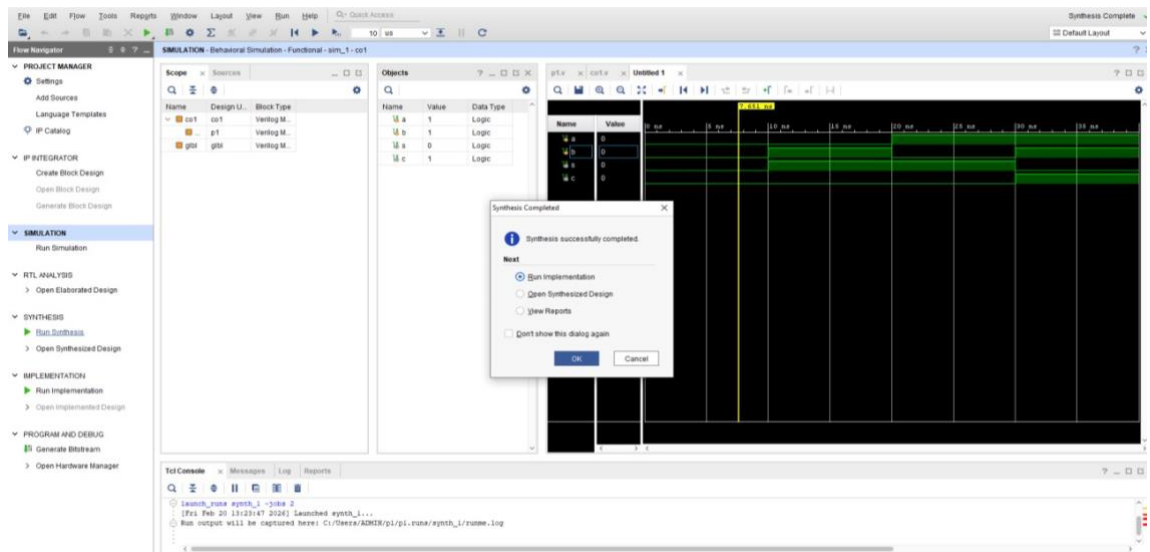


Fig 5.3: Interface of Synthesis Implementation in Xilinx Vivado

- After running Run Synthesis, wait until the process completes.
- When the Synthesis Completed dialog appears, select Run Implementation.
- Click OK to start the implementation process.

Step 15: Complete Implementation:

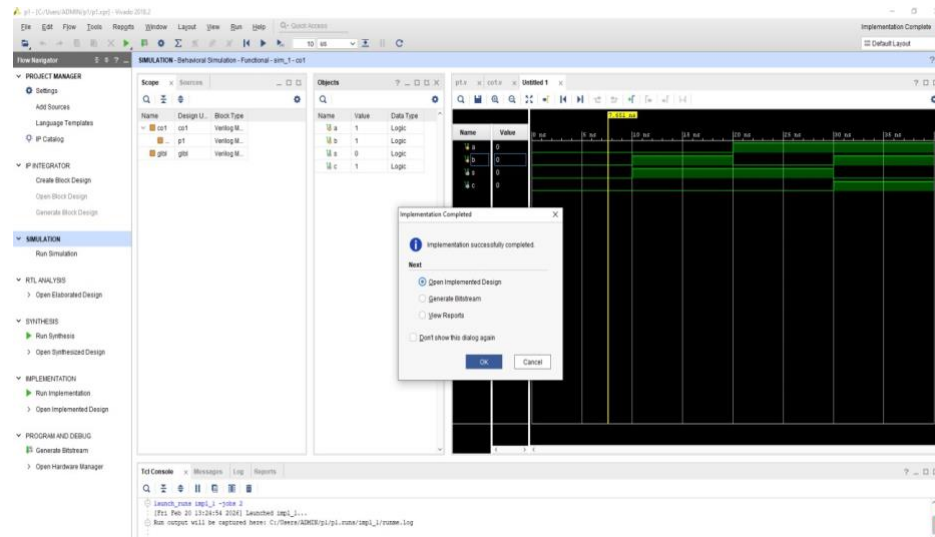


Fig 5.4: Interface of Complete Implementation in Xilinx Vivado

- Wait for the Implementation process to complete. When the Implementation Completed dialog appears, choose Open Implemented
- Design (optional for viewing layout) or proceed to generate the bitstream in the next step.
- Click OK

Step 16: Change Layout to Floor-planning View:

- Open Implemented Design
- After the successful completion of Implementation:
- In the Flow Navigator, click on Open Implemented Design.
- The implemented device layout will be displayed.
- Verify that the status shows Implementation Complete

Step 17: Change Layout to Floor-planning View

- After implementation is completed successfully, the device layout can be viewed for pin and resource verification.
- In the top-right corner of the Vivado window, locate the Layout dropdown menu.
- Click on the dropdown menu that currently shows Default Layout.
- From the available options, select Floorplanning.

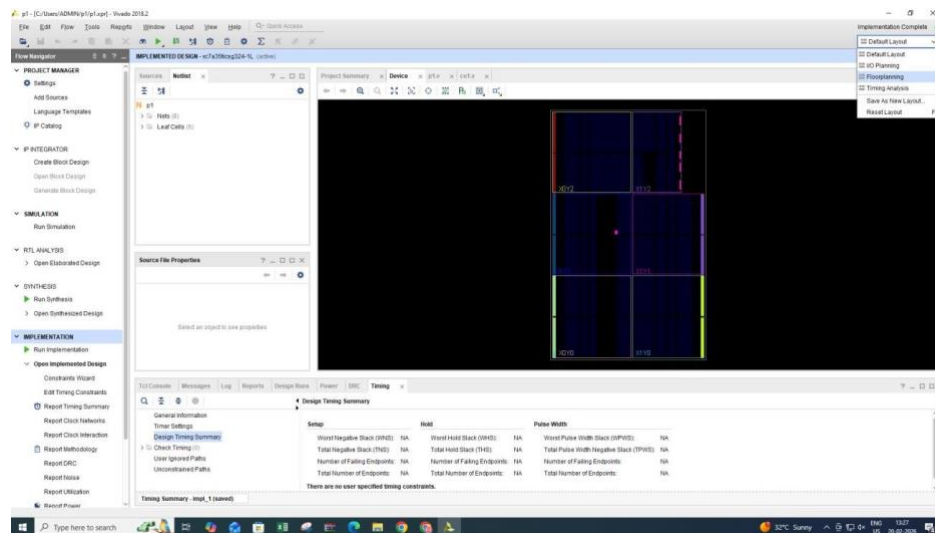
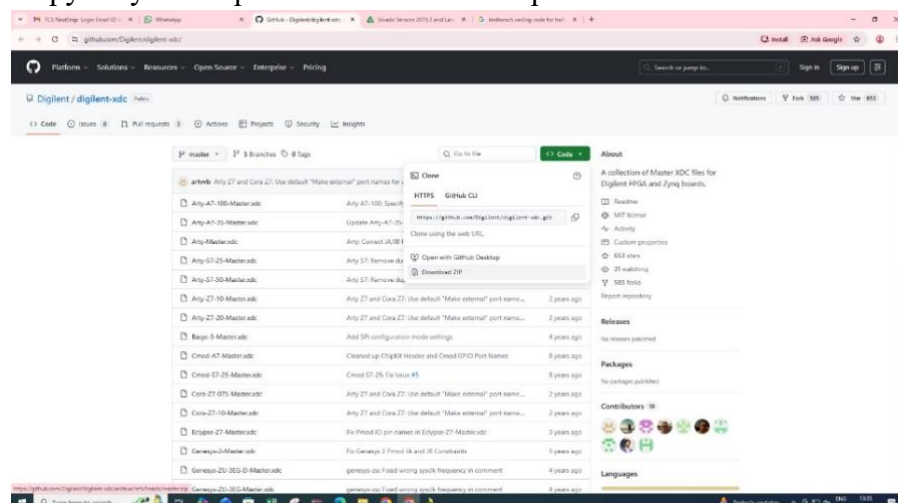


Fig 5.3: Interface of Changing Layout to Floor-planning View in Xilinx Vivado

Step 18: Extract the Required XDC File

- After successful completion of implementation, the next step is to assign the FPGA board pins to the input and output ports.
- Open a web browser.
- Go to the Digilent GitHub repository: <https://github.com/Digilent/digilent-xdc>
- Click on the Code button.
- Select Download ZIP.
- Extract the downloaded ZIP file.
- From the extracted folder, open the XDC file corresponding to the Zedboard (for example: Zedboard-Master.xdc).
- Open the file using Notepad or any text editor.
- Copy only the required switch and LED pin constraint lines from this file.



- Keep file location as Local to Project.
- Click OK.

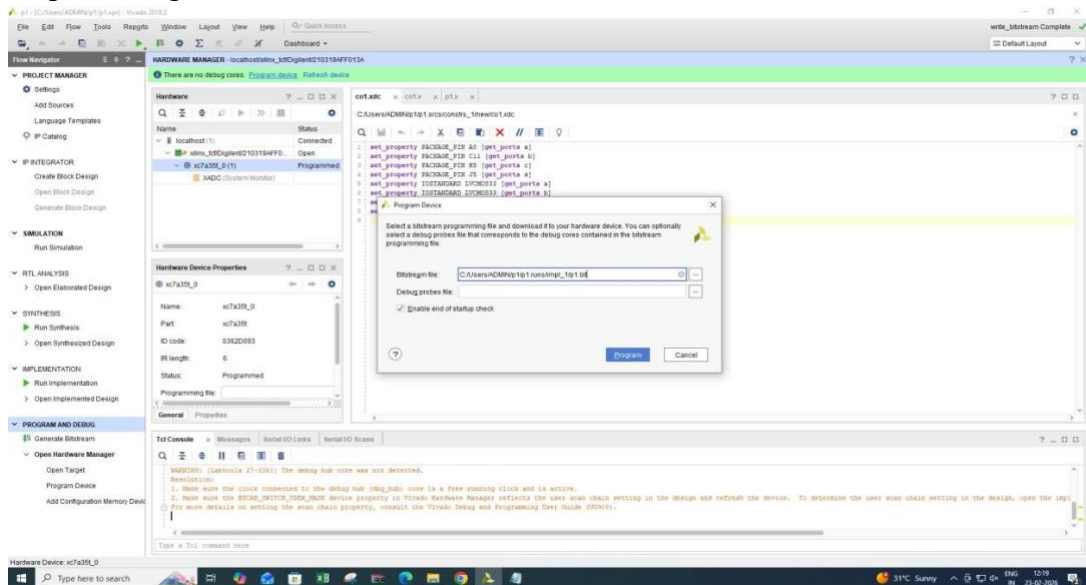
Step 22: Generate Bitstream

- In the Flow Navigator, click Generate Bitstream under Program and Debug.
- In the Launch Runs dialog, keep default options and click OK.
- Wait until the message “Bitstream Generation successfully completed” appears.
- Select Open Hardware Manager and click OK

Step 23: Open Hardware Target (Auto Connect)

- In Hardware Manager, click Open Target.
- Select Auto Connect to automatically connect to the local hardware server.

Step 24: Program the FPGA Device



- In the Hardware window, right-click the detected device (e.g., xc7a35t_0).
- Click Program Device.
- Ensure the correct Bitstream file (.bit) is selected automatically.
- Click Program to download the design to the FPGA.

5.4 Simulation and Verification

The Verilog design is simulated using standard FPGA tools such as Xilinx Vivado.

Simulation Steps

- Apply different moisture values
- Vary water level input
- Observe motor output

Expected Results

- Motor ON when soil is dry
- Motor OFF otherwise

Chapter- 6: RESULTS & ANALYSIS

6.1 Introduction

This chapter presents the simulation results and performance evaluation of the proposed FPGA-Based Automated Water Management System. The primary objective is to validate the functionality of the control logic implemented using Verilog Hardware Description Language (HDL) and to analyze the system behavior under various environmental conditions.

The results are obtained through simulation and practical implementation on FPGA hardware. Simulation plays a crucial role in verifying the correctness of the design before deploying it on physical hardware. It helps in identifying logical errors, timing issues, and functional inconsistencies.

The system is evaluated based on key parameters such as:

- Accuracy of moisture detection
- Correctness of motor control decisions
- Response time to input changes
- Reliability under different conditions

The outputs are analyzed using waveform representations, which provide a visual understanding of how input signals affect system behavior. Additionally, hardware testing results are also considered to validate real-time performance.

This chapter also includes performance analysis, comparison with existing systems, and identification of limitations.

6.1.1 Procedure for FPGA Implementation

The implementation of the proposed FPGA-Based Automated Water Management System was carried out using the Zynq-7000 SoC on the ZedBoard platform. The process involves multiple stages, including design entry, simulation, synthesis, hardware configuration, and testing. The following steps describe the complete procedure followed for FPGA implementation.

Step 1: Design Entry Using Verilog HDL

The control logic of the irrigation system was designed using Verilog Hardware Description Language (HDL). The design includes modules for processing sensor inputs, comparing them with predefined threshold values, and generating control signals for the irrigation motor.

The design was structured in a modular manner to improve readability and scalability. Each module performs a specific function such as input handling, decision-making, and output generation.

Step 2: Simulation and Functional Verification

The Verilog design was simulated using Xilinx Vivado to verify its functionality before hardware implementation. Different input conditions such as low soil moisture, high soil moisture, and water level variations were applied.

Simulation waveforms were analyzed to ensure that:

- The system responds correctly to input changes
- The motor control signal is generated accurately
- No logical errors exist in the design

Step 3: Synthesis of the Design

After successful simulation, the design was synthesized using Vivado. Synthesis converts the HDL code into a gate-level netlist that can be implemented on the FPGA hardware.

During this step:

- Logic elements are mapped to FPGA resources
- Optimization is performed for speed and area
- Resource utilization is analyzed

Step 4: Implementation and Bitstream Generation

The synthesized design is then implemented on the FPGA by performing placement and routing. This step ensures that all logic components are properly connected within the FPGA architecture.

Once implementation is completed, a bitstream file (.bit) is generated. This file contains the configuration data required to program the FPGA.

Step 5: Hardware Programming Using ZedBoard

The generated bitstream is uploaded to the ZedBoard FPGA using the Vivado Hardware Manager. The FPGA is programmed via JTAG interface, enabling it to execute the designed control logic.

After programming, the FPGA becomes ready to process real-time sensor inputs and control the irrigation system.

Step 6: Interfacing with Arduino and LoRa Modules

The Arduino Uno is used to read analog sensor data and convert it into digital form. This data is transmitted wirelessly using the LoRa transmitter module.

At the receiver side, the LoRa module connected to the FPGA receives the data and forwards it to the processing unit. Proper interfacing ensures reliable communication between all system components.

Step 7: Testing and Validation

The implemented system was tested under different conditions to validate its performance. Sensor inputs were varied to simulate different soil moisture and water level conditions.

The following observations were made:

The FPGA responds instantly to input changes

The motor is activated only when required

The system operates reliably without delay

Step 8: Real-Time Operation

Finally, the system was operated in real-time conditions to evaluate its practical performance. The FPGA continuously monitored sensor inputs and controlled the motor automatically.

The successful implementation demonstrates the effectiveness of FPGA in real-time irrigation control applications.

6.2 Simulation Setup

The Verilog design was simulated using Xilinx Vivado, targeting the Zynq-7000 SoC platform.

Simulation Inputs

- Soil moisture value (digital input from ADC via Arduino Uno)
- Water level signal (binary input)

Simulation Outputs

- Motor control signal (ON/OFF)

Different combinations of inputs were applied to test the system under various conditions.

Table 6.1: Simulation Parameters and System Response

S.No	Soil Moisture Level	Water Level Status	Motor Control Output	Description	Reference
1	Low	Available	ON	Soil is dry, water available → Irrigation ON	[1][3]
2	Low	Not Available	OFF	Soil is dry, but no water → Motor OFF	[1]
3	High	Available	OFF	Soil is wet → No irrigation needed	[4]
4	High	Not Available	OFF	Soil is wet and no water → No action required	[5]

6.3 Simulation Waveforms

The simulation waveforms provide a graphical representation of the system behavior over time. They show how the input signals (soil moisture and water level) influence the output signal (motor control).

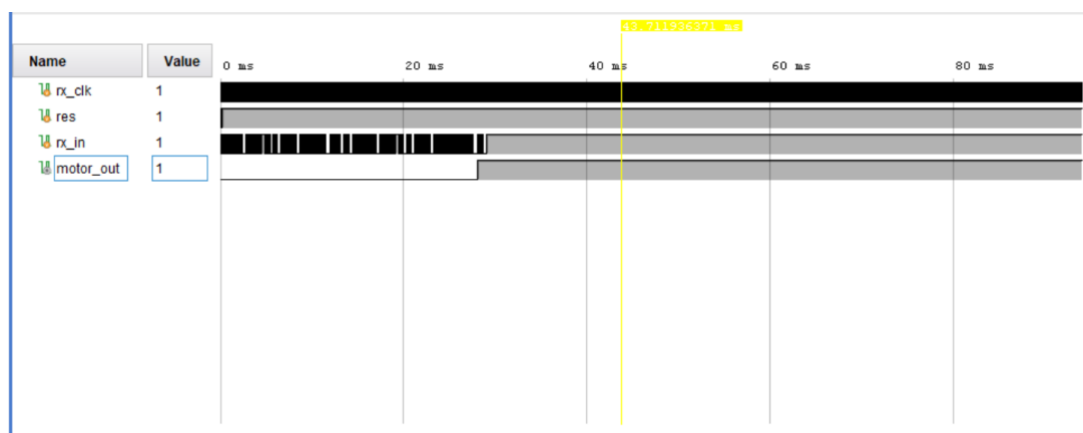


Fig 6.1: Simulation Waveform Showing Motor Control Output Based on Input Signal Conditions

In the waveform:

- Input signals are plotted along the time axis
- Output signal changes based on input conditions
- Transitions indicate system response

When the soil moisture level falls below the threshold, the motor control signal transitions to HIGH, indicating that the motor is turned ON. Conversely, when the moisture level exceeds the threshold, the motor control signal becomes LOW, turning OFF the motor.

Waveforms also demonstrate the real-time response capability of the FPGA, where output changes occur almost instantly after input changes.

The waveform illustrates the relationship between input signals (moisture) and output signal (motor control).

Table 6.2: Waveform Analysis of Input–Output Behavior

S.No	Soil Moisture Condition	Water Level Status	Motor Control Signal	Waveform Observation	Reference
1	Below Threshold	Available	HIGH (ON)	Output rises immediately after input change	[1][3]
2	Below Threshold	Not Available	LOW (OFF)	No activation due to insufficient water	[1]
3	Above Threshold	Available	LOW (OFF)	Output remains LOW (no irrigation required)	[4]
4	Above Threshold	Not Available	LOW (OFF)	System remains inactive	[5]

6.4 FPGA Implementation

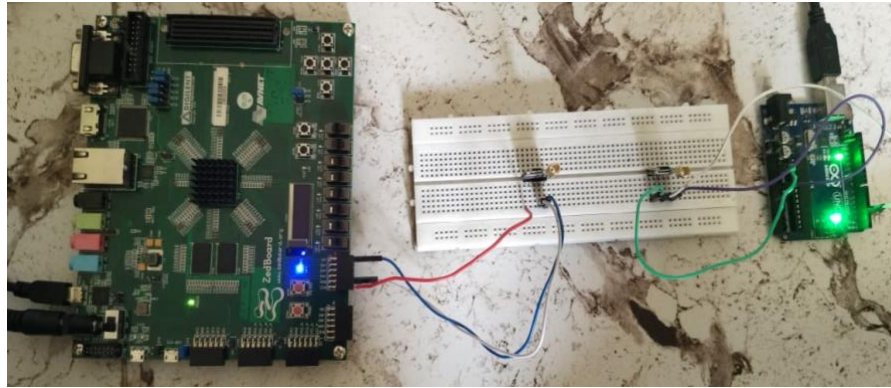


Fig 6.2: Experimental Setup of FPGA-Based Irrigation System with ZedBoard, Arduino, and Sensor Interface

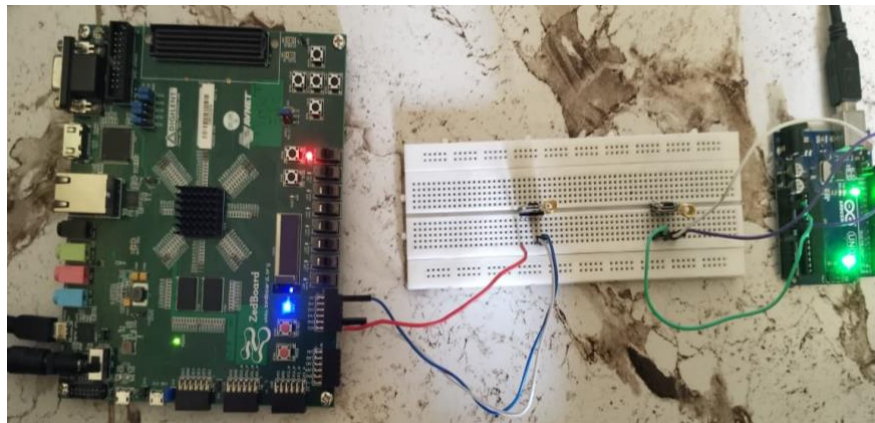


Fig 6.3: Experimental Setup Showing Motor ON Condition Based on Sensor Input

The FPGA based Water Management System for Smart Agriculture was successfully implemented and tested using the ZedBoard to operate the irrigation motor from the input of the soil moisture sensor. The main aim of the system was to automate the irrigation, providing water to the soil only when required. It was accomplished by processing sensor input and comparing it with pre-set threshold values to adjust the motor state.

Motor OFF – Sufficient Moisture: In this test, the water level was higher than the set threshold. The motor was still OFF, which meant that the irrigation system properly identified that no irrigation was required. This outcome verified that the system could save water by avoiding unnecessary irrigation.

Motor ON – Low Soil Moisture: During this test, the soil moisture was above threshold.

The motor turned on as expected, starting the irrigation process to replenish the optimal soil conditions. This result proved the system's capability to sense low moisture and induce irrigation accordingly.

6.5 Performance Analysis

The proposed system demonstrates the following performance characteristics:

Real-Time Response

The FPGA-based design ensures an immediate response to changes in sensor inputs due to parallel processing capability.

Accuracy

The threshold-based decision-making provides accurate control over irrigation, reducing chances of overwatering or underwatering.

Reliability

The system ensures safe operation by incorporating water level checks, preventing motor damage.

Efficiency

- Reduces water wastage
- Minimizes manual intervention
- Ensures optimal irrigation

6.6 Comparison with Existing Systems

Compared to microcontroller-based systems:

- Faster response due to FPGA parallelism
- Better handling of real-time conditions
- More reliable decision-making

Compared to IoT systems:

- No dependency on internet
- No latency in decision-making

Table 6.3: Comparison of FPGA-Based System with Existing Systems

S.No	Parameter	Microcontroller-Based System	IoT-Based System	FPGA-Based System (Proposed)	Reference
1	Processing Type	Sequential processing	Sequential + Network dependent	Parallel processing	[6][8]
2	Response Time	Moderate	High latency due to network	Very fast (real-time response)	[1][3]
3	Real-Time Capability	Limited	Affected by connectivity	Excellent real-time performance	[4]
4	Reliability	Moderate	Depends on internet stability	High reliability (hardware-based control)	[5]
5	Dependency	Standalone	Requires internet/cloud	Independent of internet	[1]
6	Decision Speed	Slower	Delayed due to data transmission	Instant decision-making	[3][4]
7	Scalability	Limited	High	High with efficient hardware utilization	[4]

6.7 Limitations Observed

Although the proposed system demonstrates efficient performance, certain limitations were observed during testing.

- Fixed threshold values may not be suitable for all soil types
- Sensor accuracy may affect decision-making
- Limited remote monitoring capability without advanced IoT integration

These limitations can be addressed in future work by incorporating adaptive algorithms, high-precision sensors, and cloud-based monitoring systems.

Chapter- 7: LIMITATIONS

7.1 Hardware and Sensor Limitations

The performance of the proposed FPGA-based automated irrigation system is highly dependent on the accuracy and reliability of hardware components, particularly the sensors and interfacing modules. Although the system is designed to operate efficiently, certain limitations arise due to practical constraints in hardware implementation.

One of the major limitations is related to the soil moisture sensor. The sensor operates based on electrical conductivity, which varies with soil type, temperature, and environmental conditions. As a result, the sensor readings may not always accurately represent the actual moisture content of the soil. For instance, sandy soil and clay soil exhibit different conductivity characteristics, leading to variations in sensor output even under similar moisture conditions.

Another important factor is sensor calibration. Proper calibration is essential to ensure accurate readings; however, in real-world applications, calibration may not always be precise. Improper calibration can result in incorrect irrigation decisions, such as unnecessary watering or insufficient irrigation.

Table 8.1: Hardware Limitations of Proposed FPGA-Based Irrigation System

S.No	Component/Factor	Limitation Description	Impact on System Performance
1	Soil Moisture Sensor	Output varies with soil type, temperature, and environmental conditions	Inaccurate moisture readings
2	Soil Type Variation	Different soils (sand, clay) produce different conductivity values	Inconsistent sensor output
3	Sensor Calibration	Calibration may not be precise in real-world conditions	Incorrect irrigation decisions
4	Arduino (ADC Unit)	Requires proper analog-to-digital conversion and interfacing	Possible data conversion errors

S.No	Component/Factor	Limitation Description	Impact on System Performance
5	LoRa Module	Needs proper configuration for reliable communication	Data transmission delays or errors
6	Hardware Integration	Multiple modules must be properly synchronized	Increased system complexity
7	Power Consumption	Multiple components increase energy requirements	Reduced efficiency in long-term operation
8	Maintenance	Difficult to maintain in remote agricultural environments	Reduced reliability over time

In addition to sensor limitations, the use of external modules such as Arduino for ADC conversion and LoRa for communication introduces complexity in hardware integration. These modules require proper configuration and interfacing, and any misalignment can lead to data transmission errors or system malfunction.

Furthermore, the dependency on multiple hardware components increases system complexity and power requirements. This may affect long-term reliability, especially in remote agricultural environments where maintenance is limited.

7.2 System-Level and Functional Limitations

Apart from hardware constraints, the proposed system also exhibits certain limitations at the system and functional level. These limitations mainly arise due to the use of fixed control logic and limited environmental awareness.

The system operates based on predefined threshold values for soil moisture. While this approach simplifies the design and ensures basic automation, it lacks adaptability to varying environmental conditions. Different crops require different moisture levels, and fixed thresholds may not be suitable for all scenarios. Additionally, environmental factors such as temperature, humidity, and rainfall are not considered in the current system.

Another limitation is the absence of a user interface or mobile notification system. The system functions autonomously without providing real-time feedback to the user. This

restricts the ability of farmers to monitor system performance or respond to unexpected situations, especially when the system is deployed in remote locations.

Although LoRa communication is used, it is primarily for data transmission and does not provide a complete monitoring solution. There is no provision for alerts or remote control through mobile applications.

Moreover, the system lacks intelligent decision-making capabilities. It does not utilize advanced techniques such as machine learning or predictive analysis to optimize irrigation. As a result, the system cannot adapt dynamically to changing environmental conditions or historical data patterns.

Chapter- 8: SUMMARY, CONCLUSION & FUTURE SCOPE

8.1 Summary of the Work

This project presented the design and implementation of an FPGA-Based Automated Water Management System for Smart Agriculture, aimed at improving irrigation efficiency through real-time monitoring and control. The system integrates soil moisture and water level sensors to continuously monitor environmental conditions and make intelligent decisions regarding irrigation.

An Arduino Uno was used to convert analog sensor signals into digital form, which were then processed by the Zynq-7000 SoC. The control logic implemented using Verilog HDL enabled real-time decision-making to control the irrigation motor based on predefined threshold values.

The system was designed using a modular approach, including input acquisition, threshold comparison, control logic, and output modules. Simulation results verified the correct functioning of the system under different conditions, ensuring efficient and reliable irrigation control.

8.2 Conclusions

Based on the design, implementation, and analysis of the system, the following conclusions can be drawn:

1. The proposed system successfully automates irrigation based on real-time soil moisture and water level conditions.
2. The use of FPGA enables high-speed processing and real-time response, making the system more efficient compared to traditional microcontroller-based systems.
3. The integration of sensors ensures accurate monitoring of environmental parameters, reducing water wastage and improving crop productivity.
4. The system provides reliable operation by incorporating safety mechanisms such as water level monitoring to prevent motor damage.
5. The modular design of the system allows easy scalability and future enhancements.

Overall, the project demonstrates that FPGA-based control systems can play a significant role in modernizing agricultural practices and promoting sustainable water management.

8.3 Future Scope

Although the proposed system achieves efficient irrigation automation, several enhancements can be incorporated to further improve its functionality and applicability:

IoT-Based Monitoring and Control

Integration with IoT platforms can enable real-time data logging, cloud-based analysis, and remote access through mobile applications, allowing farmers to monitor field conditions from anywhere.

Multi-Sensor Integration

Additional sensors such as temperature, humidity, and pH sensors can be incorporated to provide a more comprehensive analysis of soil and environmental conditions.

Adaptive Irrigation Using AI

Artificial intelligence and machine learning algorithms can be implemented to dynamically adjust irrigation based on historical data, weather conditions, and crop requirements.

Solar-Powered System

The system can be made energy-efficient by integrating renewable energy sources such as solar panels, making it suitable for remote and off-grid locations.

Mobile Application Interface

A user-friendly mobile application can be developed to provide real-time alerts, system status updates, and manual control options.

The proposed FPGA-based automated irrigation system provides an effective solution to address the challenges of water management in agriculture. By combining real-time monitoring, high-speed processing, and automation, the system contributes to efficient resource utilization and sustainable farming practices. With further enhancements, the system has the potential to evolve into a fully intelligent and connected smart agriculture solution.

REFERENCES

- [1] Joaquín Gutiérrez, Juan Francisco Villa-Medina, Alejandra Nieto-Garibay, and Miguel Ángel Porta-Gándara, “Automated Irrigation System Using a Wireless Sensor Network and GPRS Module,” *IEEE Transactions on Instrumentation and Measurement*, vol. 63, no. 1, January 2014.
- [2] Jia Uddin, S.M. Taslim Reza, Qader Newaz, Jamal Uddin, Touhidul Islam, and Jong-Myon Kim, “Automated Irrigation System Using Solar Power” ©2012 IEEE
- [3] Joaquín Gutiérrez, Juan Francisco Villa-Medina, Alejandra Nieto Garibay, and Miguel Ángel Porta-Gándara “Automated Irrigation System Using a Wireless Sensor Network and GPRS Module ” IEEE 2013
- [4] Manish Giri, Dnyaneshwar Natha Wavhal (2013). Automated Intelligent Wireless Drip Irrigation Using Linear Programming. *International Journal of Advanced Research in Computer Engineering & Technology (IJARCET)* Volume 2, Issue 1 An
- [5] Miranda FR, Yoder RE, Wilkerson JB, Odhiambo LO (2005). autonomous controller for site-specific management of fixed irrigation systems. *Comput. Electron. Agric.*, 48:183-197.
- [6] Zazueta FS, Smajstrla AG (1992). Microcomputer-based control of irrigation systems. *Appl. Eng. Agric.*, 8(5): 593-596.
- [7] B. Astrand and A. Baerdveldt, A vision based row following system for agricultural field machinery, *Mechatronics*, vol. 15, no. 2, pp. 251-269, 2005.
- [8] Leo Louis, “WORKING PRINCIPLE OF ARDUINO AND USING IT AS A TOOL FOR STUDY AND RESEARCH”, *International Journal of Control, Automation, Communication and Systems (IJCACS)*, Vol.1, No.2, April 2016
- [9] ARDUINO.CC, “Arduino – Introduction”, 2015
[Online], Available: <http://arduino.cc/en/Guide/Introduction>. [Accessed: 25- Feb - 2015]
- [10] Arduino.cc, 'Arduino - Products', 2015. [Online]. Available: <http://arduino.cc/en/Main/Products>. [Accessed: 25- Feb- 2015].
- [11] www.arduino.cc.